

CHAPTER 5

Superscalar Techniques

CHAPTER OUTLINE

- 5.1 Instruction Flow Techniques
- 5.2 Register Data Flow Techniques
- 5.3 Memory Data Flow Techniques
- 5.4 Summary

References

Homework Problems

In Chapter 4 we focused on the structural, or organizational, design of the superscalar pipeline and dealt with issues that were somewhat independent of the specific types of instructions being processed. In this chapter we focus more on the dynamic behavior of a superscalar processor and consider techniques that deal with specific types of instructions. The ultimate performance goal of a superscalar pipeline is to achieve maximum throughput of instruction processing. It is convenient to view instruction processing as involving three component flows of instructions and/or data, namely, *instruction flow*, *register data flow*, and *memory data flow*. This partitioning into three flow paths is similar to that used in Mike Johnson's 1991 textbook entitled *Superscalar Microprocessor Design* [Johnson, 1991]. The overall performance objective is to maximize the volumes in all three of these flow paths. Of course, what makes this task interesting is that the three flow paths are not independent and their interactions are quite complex. This chapter classifies and presents superscalar microarchitecture techniques based on their association with the three flow paths.

The three flow paths correspond roughly to the processing of the three major types of instructions, namely, branch, ALU, and load/store instructions. Consequently, maximizing the throughput of the three flow paths corresponds to the minimizing of the branch, ALU, and load penalties.

1. *Instruction flow*. Branch instruction processing.
2. *Register data flow*. ALU instruction processing.
3. *Memory data flow*. Load/store instruction processing.

This chapter uses these three flow paths as a convenient framework for presenting the plethora of microarchitecture techniques for optimizing the performance of modern superscalar processors.

5.1 Instruction Flow Techniques

We present instruction flow techniques first because these deal with the early stages, e.g., the fetch and decode stages, of a superscalar pipeline. The throughput of the early pipeline stages will impose an upper bound on the throughput of all subsequent stages. For contemporary pipelined processors, the traditional partitioning of a processor into control path and data path is no longer clear or effective. Nevertheless, the early pipeline stages along with the branch execution unit can be viewed as corresponding to the traditional control path whose primary function is to enforce the control flow semantics of a program. The primary goal for all instruction flow techniques is to maximize the supply of instructions to the superscalar pipeline subject to the requirements of the control flow semantics of a program.

5.1.1 Program Control Flow and Control Dependences

The control flow semantics of a program are specified in the form of the control flow graph (CFG), in which the nodes represent basic blocks and the edges represent the transfer of control flow between basic blocks. Figure 5.1(a) illustrates a CFG with four basic blocks (dashed-line rectangles), each containing a number of instructions (ovals). The directed edges represent control flows between basic blocks. These edges are induced by conditional branch instructions (diamonds). The run-time execution of a program entails the dynamic traversal of the nodes and edges of its CFG. The actual path of traversal is dictated by the branch instructions and their branch conditions which can be dependent on run-time data.

The basic blocks, and their constituent instructions, of a CFG must be stored in sequential locations in the program memory. Hence the partial ordered basic blocks in a CFG must be arranged in a total order in the program memory. In mapping a CFG to linear consecutive memory locations, additional unconditional branch instructions must be added, as illustrated in Figure 5.1(b). The mapping of the CFG to a linear program memory facilitates an implied sequential flow of control along the sequential memory locations during program execution. However, the encounter of both conditional and unconditional branches at run time induces deviations from this implied sequential control flow and the consequent disruptions to the sequential fetching of instructions. Such disruptions cause stalls in the instruction fetch stage of the pipeline and reduce the overall instruction fetching bandwidth. Subroutine jump and return instructions also induce similar disruptions to the sequential fetching of instructions.

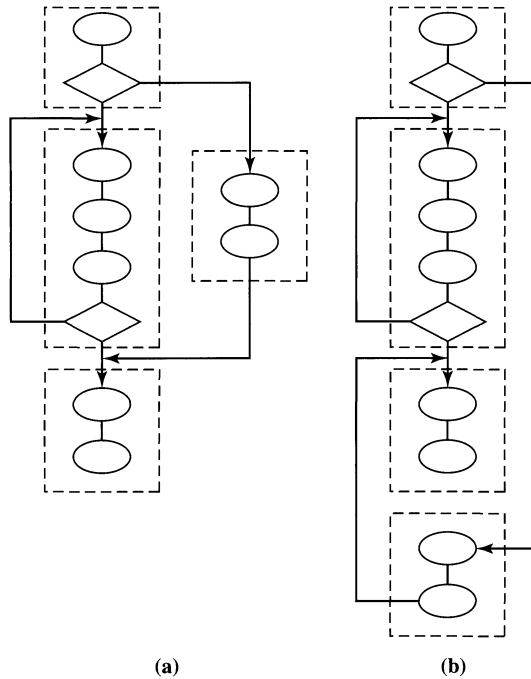


Figure 5.1

Program Control Flow: (a) The Control Flow Graph (CFG);
(b) Mapping the CFG to Sequential Memory Locations.

5.1.2 Performance Degradation Due to Branches

A pipelined machine achieves its maximum throughput when it is in the streaming mode. For the fetch stage, streaming mode implies the continuous fetching of instructions from sequential locations in the program memory. Whenever the control flow of the program deviates from the sequential path, potential disruption to the streaming mode can occur. For unconditional branches, subsequent instructions cannot be fetched until the target address of the branch is determined. For conditional branches, the machine must wait for the resolution of the branch condition, and if the branch is to be taken, it must further wait until the target address is available. Figure 5.2 illustrates the disruption of the streaming mode by branch instructions. Branch instructions are executed by the branch functional unit. For a conditional branch, it is not until it exits the branch unit and when both the branch condition and the branch target address are known that the fetch stage can correctly fetch the next instruction.

As Figure 5.2 illustrates, this delay in processing conditional branches incurs a penalty of three cycles in fetching the next instruction, corresponding to the traversal of the decode, dispatch, and execute stages by the conditional branch. The actual lost-opportunity cost of three stalled cycles is not just three empty instruction

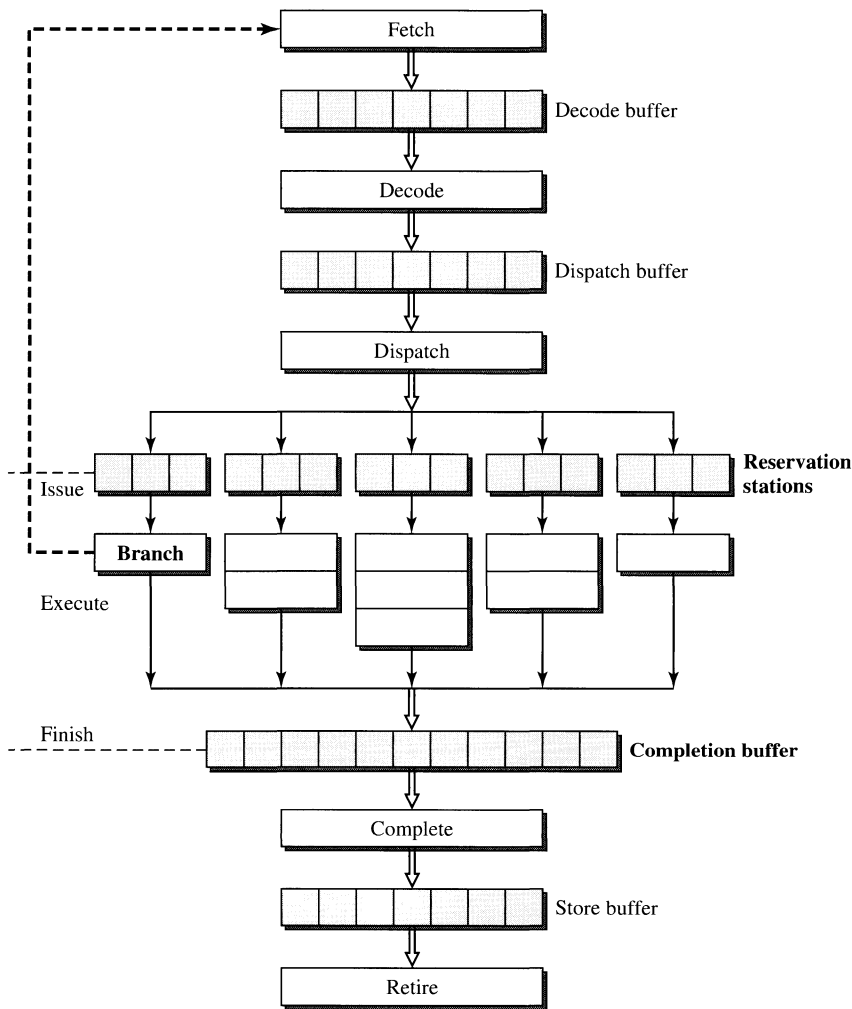
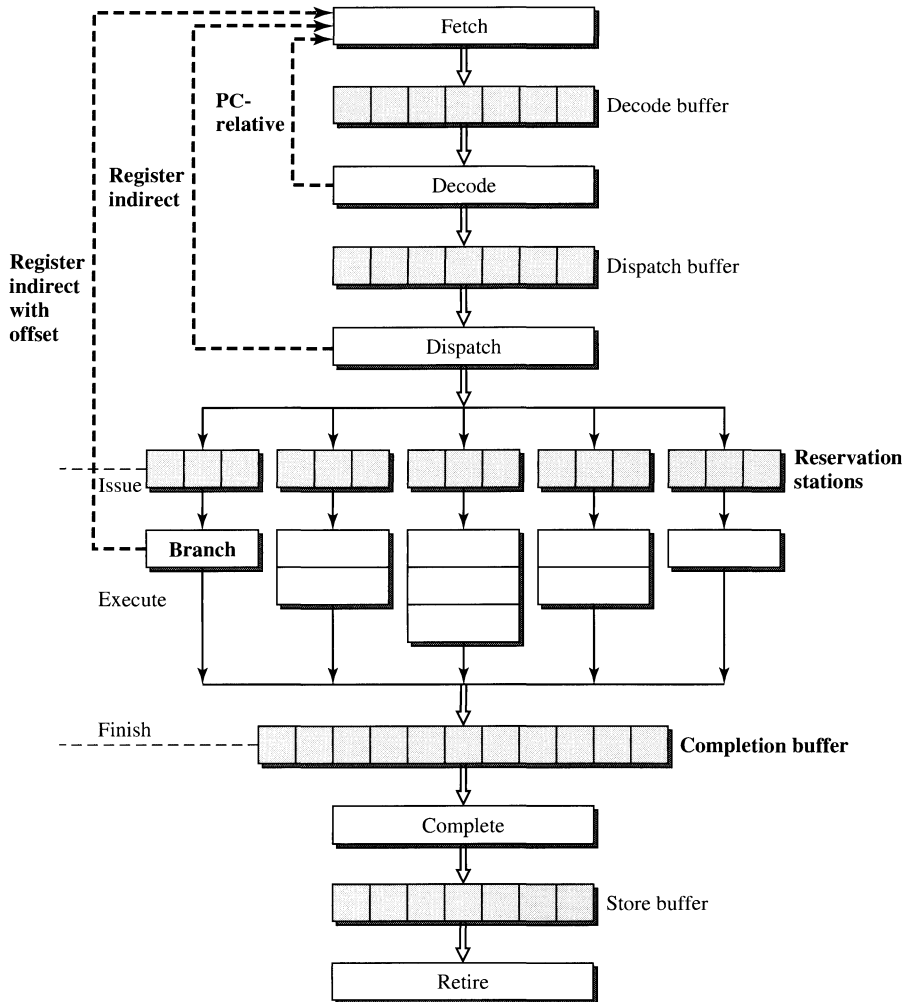


Figure 5.2
Disruption of Sequential Control Flow by Branch Instructions.

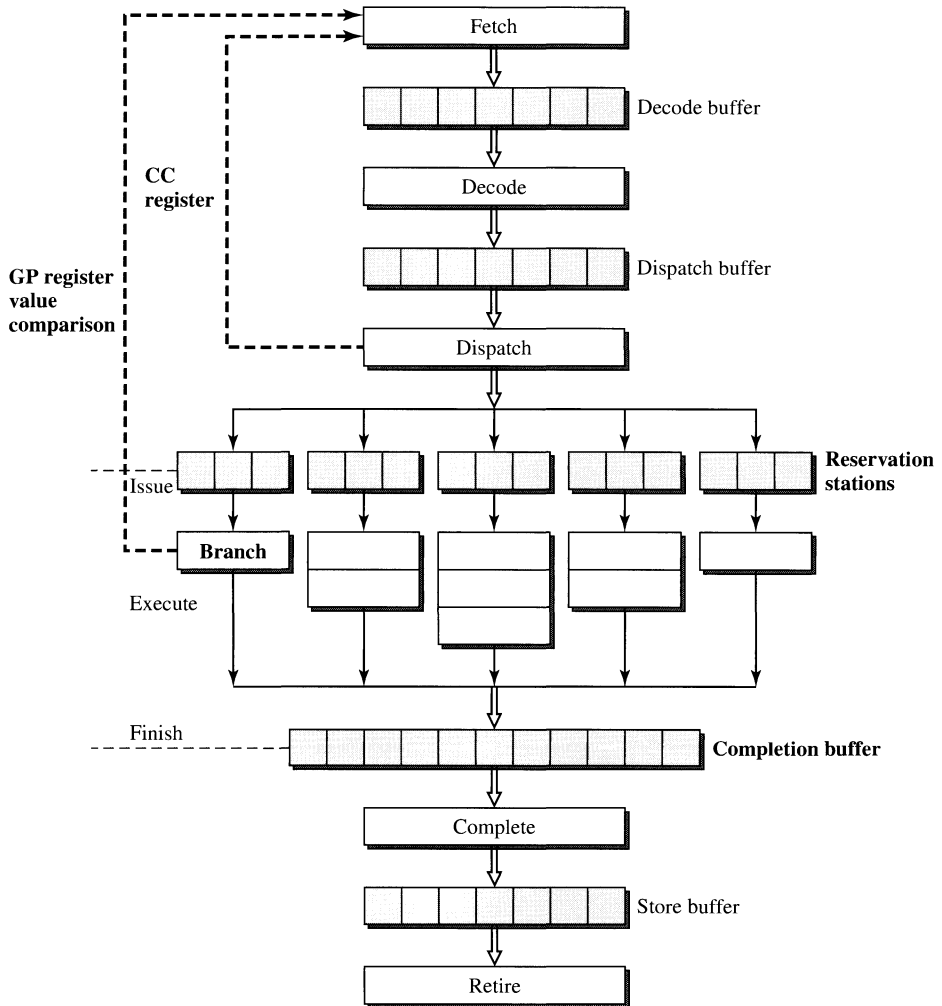
slots as in the scalar pipeline, but the number of empty instruction slots must be multiplied by the width of the machine. For example, for a four-wide machine the total penalty is 12 instruction “bubbles” in the superscalar pipeline. Also recall from Chapter 1, that such pipeline stall cycles effectively correspond to the *sequential bottleneck* of Amdahl’s law and rapidly and significantly reduce the actual performance from the potential peak performance.

For conditional branches, the actual number of stalled or penalty cycles can be dictated by either target address generation or condition resolution. Figure 5.3 illustrates the potential cycles that can be incurred by target address generation. The

**Figure 5.3**

Branch Target Address Generation Penalties.

actual number of penalty cycles is determined by the addressing modes of the branch instructions. For the PC-relative addressing mode, the branch target address can be generated during the fetch stage, resulting in a penalty of one cycle. If the register indirect addressing mode is used, the branch instruction must traverse the decode stage to access the register. In this case a two-cycle penalty is incurred. For register indirect with an offset addressing mode, the offset must be added after register access and a total three-cycle penalty can result. For unconditional branches, only the penalty due to target address generation is of concern. For conditional branches, branch condition resolution latency must also be considered.

**Figure 5.4**

Branch Condition Resolution Penalties.

Different methods for performing condition resolution can also lead to different penalties. Figure 5.4 illustrates two possible penalties. If condition code registers are used, and assuming that the relevant condition code register is accessed during the dispatch stage, then a penalty of two cycles will result. If the ISA permits the comparison of two general-purpose registers to generate the branch condition, then one more cycle is needed to perform an ALU operation on the contents of the two registers. This will result in a penalty of three cycles. For a conditional branch, depending on the addressing mode and condition resolution method used, either one of the penalties may be the critical one. For example, even if the PC-relative

addressing mode is used, a conditional branch that must access a condition code register will still incur a two-cycle penalty instead of the one-cycle penalty for target address generation.

Maximizing the volume of the instruction flow path is equivalent to maximizing the sustained instruction fetch bandwidth. To do this, the number of stall cycles in the fetch stage must be minimized. Recall that the total lost-opportunity cost is equal to the product of the number of penalty cycles and the width of a machine. For an n -wide machine each stalled cycle is equal to fetching n no-op instructions. The primary aim of instruction flow techniques is to minimize the number of such fetch stall cycles and/or to make use of these cycles to do potentially useful work. The current dominant approach to accomplishing this is via branch prediction which is the subject of Section 5.1.3.

5.1.3 Branch Prediction Techniques

Experimental studies have shown that the behavior of branch instructions is highly predictable. A key approach to minimizing branch penalty and maximizing instruction flow throughput is to speculate on both branch target addresses and branch conditions of branch instructions. As a static branch instruction is repeatedly executed at run time, its dynamic behavior can be tracked. Based on its past behavior, its future behavior can be effectively predicted. Two fundamental components of branch prediction are *branch target speculation* and *branch condition speculation*. With any speculative technique, there must be mechanisms to validate the prediction and to safely recover from any mispredictions. Branch misprediction recovery will be covered in Section 5.1.4.

Branch target speculation involves the use of a branch target buffer (BTB) to store previous branch target addresses. BTB is a small cache memory accessed during the instruction fetch stage using the instruction fetch address (PC). Each entry of the BTB contains two fields: the branch instruction address (BIA) and the branch target address (BTA). When a static branch instruction is executed for the first time, an entry in the BTB is allocated for it. Its instruction address is stored in the BIA field, and its target address is stored in the BTA field. Assuming the BTB is a fully associative cache, the BIA field is used for the associative access of the BTB. The BTB is accessed concurrently with the accessing of the I-cache. When the current PC matches the BIA of an entry in the BTB, a hit in the BTB results. This implies that the current instruction being fetched from the I-cache has been executed before and is a branch instruction. When a hit in the BTB occurs, the BTA field of the hit entry is accessed and can be used as the next instruction fetch address if that particular branch instruction is predicted to be taken; see Figure 5.5.

By accessing the BTB using the branch instruction address and retrieving the branch target address from the BTB all during the fetch stage, the speculative branch target address will be ready to be used in the next machine cycle as the new instruction fetch address if the branch instruction is predicted to be taken. If the branch instruction is predicted to be taken and this prediction turns out to be correct, then the branch instruction is effectively executed in the fetch stage, incurring no branch penalty. The nonspeculative execution of the branch instruction is still

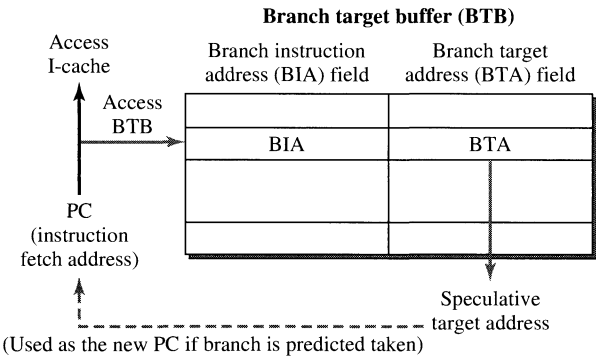
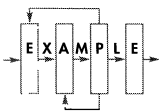


Figure 5.5
Branch Target Speculation Using a Branch Target Buffer.

performed for the purpose of validating the speculative execution. The branch instruction is still fetched from the I-cache and executed. The resultant target address and branch condition are compared with the speculative version. If they agree, then correct prediction was made; otherwise, misprediction has occurred and recovery must be initiated. The result from the nonspeculative execution is also used to update the content, i.e., the BTA field, of the BTB.

There are a number of ways to do branch condition speculation. The simplest form is to design the fetch hardware to be biased for not taken, i.e., to always predict not taken. When a branch instruction is encountered, prior to its resolution, the fetch stage continues fetching down the fall-through path without stalling. This form of minimal branch prediction is easy to implement but is not very effective. For example, many branches are used as loop closing instructions, which are mostly taken during execution except when exiting loops. Another form of prediction employs software support and can require ISA changes. For example, an extra bit can be allocated in the branch instruction format that is set by the compiler. This bit is used as a hint to the hardware to perform either predict not taken or predict taken depending on the value of this bit. The compiler can use branch instruction type and profiling information to determine the most appropriate value for this bit. This allows each static branch instruction to have its own specified prediction. However, this prediction is static in the sense that the same prediction is used for all dynamic executions of the branch. Such static software prediction technique is used in the Motorola 88110 [Diefendorf and Allen, 1992]. A more aggressive and dynamic form of prediction makes prediction based on the branch target address offset. This form of prediction first determines the relative offset between the address of the branch instruction and the address of the target instruction. A positive offset will trigger the hardware to predict not taken, whereas a negative offset, most likely indicating a loop closing branch, will trigger the hardware to predict taken. This branch offset-based technique is used in the original IBM RS/6000 design and has been adopted by other machines as well [Grohoski, 1990; Oehler and Groves, 1990]. The most common branch condition speculation technique



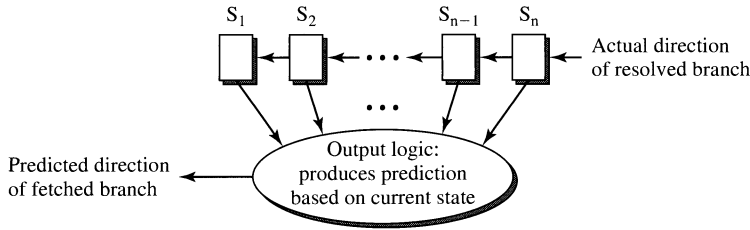


Figure 5.6

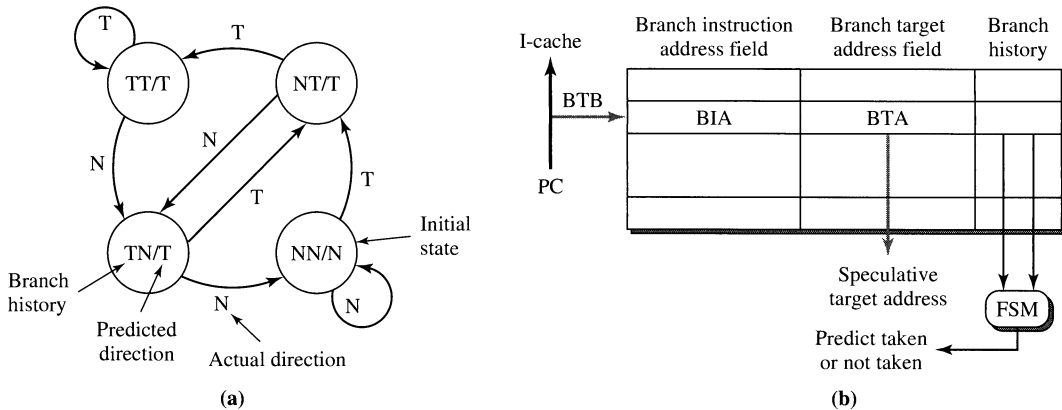
FSM Model for History-Based Branch Direction Predictors.

employed in contemporary superscalar machines is based on the history of previous branch executions.

History-based branch prediction makes a prediction of the branch direction, whether taken (T) or not taken (N), based on previously observed branch directions. This approach was first proposed by Jim Smith, who patented the technique on behalf of his employer, Control Data, and later published an important early study [Smith, 1981]. The assumption is that historical information on the direction that a static branch takes in previous executions can give helpful hints on the direction that it is likely to take in future executions. Design decisions for such type of branch prediction include how much history should be tracked and for each observed history pattern what prediction should be made. The specific algorithm for history-based branch direction prediction can be characterized by a finite state machine (FSM); see Figure 5.6. The n state variables encode the directions taken by the last n executions of that branch. Hence each state represents a particular history pattern in terms of a sequence of takens and not takens. The output logic generates a prediction based on the current state of the FSM. Essentially, a prediction is made based on the outcome of the previous n executions of that branch. When a predicted branch is finally executed, the actual outcome is used as an input to the FSM to trigger a state transition. The next state logic is trivial; it simply involves chaining the state variables into a shift register, which records the branch directions of the previous n executions of that branch instruction.

Figure 5.7(a) illustrates the FSM diagram of a typical 2-bit branch predictor that employs two history bits to track the outcome of two previous executions of the branch. The two history bits constitute the state variables of the FSM. The predictor can be in one of four states: NN, NT, TT, or TN, representing the directions taken in the previous two executions of the branch. The NN state can be designated as the initial state. An output value of either T or N is associated with each of the four states representing the prediction that would be made when a predictor is in that state. When a branch is executed, the actual direction taken is used as an input to the FSM, and a state transition occurs to update the branch history which will be used to do the next prediction.

The particular algorithm implemented in the predictor of Figure 5.7(a) is biased toward predicting branches to be taken; note that three of the four states

**Figure 5.7**

History-Based Branch Prediction: (a) A 2-Bit Branch Predictor Algorithm; (b) Branch Target Buffer with an Additional Field for Storing Branch History Bits.

predict the branch to be taken. It anticipates either long runs of N's (in the NN state) or long runs of T's (in the TT state). As long as at least one of the two previous executions was a taken branch, it will predict the next execution to be taken. The prediction will only be switched to not taken when it has encountered two consecutive N's in a row. This represents one particular branch direction prediction algorithm; clearly there are many possible designs for such history-based predictors, and many designs have been evaluated by researchers.

To support history-based branch direction predictors, the BTB can be augmented to include a history field for each of its entries. The width, in number of bits, of this field is determined by the number of history bits being tracked. When a PC address hits in the BTB, in addition to the speculative target address, the history bits are retrieved. These history bits are fed to the logic that implements the next-state and output functions of the branch predictor FSM. The retrieved history bits are used as the state variables of the FSM. Based on these history bits, the output logic produces the 1-bit output that indicates the predicted direction. If the prediction is a taken branch, then this output is used to steer the speculative target address to the PC to be used as the new instruction fetch address in the next machine cycle. If the prediction turns out to be correct, then effectively the branch instruction has been executed in the fetch stage without incurring any penalty or stalled cycle.

A classic experimental study on branch prediction was done by Lee and Smith [1984]. In this study, 26 programs from six different types of workloads for three different machines (IBM 370, DEC PDP-11, and CDC 6400) were used. Averaged across all the benchmarks, 67.6% of the branches were taken while 32.4% were not taken. Branches tend to be taken more than not taken by a ratio of 2 to 1. With static branch prediction based on the op-code type, the prediction accuracy ranged from 55% to 80% for the six workloads. Using only 1 bit of history, history-based dynamic branch prediction achieved prediction accuracies ranging from 79.7% to

96.5%. With 2 history bits, the accuracies for the six workloads ranged from 83.4% to 97.5%. Continued increase of the number of history bits brought additional incremental accuracy. However, beyond four history bits there is a very minimal increase in the prediction accuracy. They implemented a four-way set associative BTB that had 128 sets. The averaged BTB hit rate was 86.5%. Combining prediction accuracy with the BTB hit rate, the resultant average prediction effectiveness was approximately 80%.

Another experimental study was done in 1992 at IBM by Ravi Nair using the RS/6000 architecture and Systems Performance Evaluation Cooperative (SPEC) benchmarks [Nair, 1992]. This was a very comprehensive study of possible branch prediction algorithms. The goal for branch prediction is to overlap the execution of branch instructions with that of other instructions so as to achieve zero-cycle branches or accomplish *branch folding*; i.e., branches are folded out of the critical latency path of instruction execution. This study performed an exhaustive search for optimal 2-bit predictors. There are 2^{20} possible FSMs of 2-bit predictors. Nair determined that many of these machines are uninteresting and pruned the entire design space down to 5248 machines. Extensive simulations are performed to determine the optimal (achieves the best prediction accuracy) 2-bit predictor for each of the benchmarks. The list of SPEC benchmarks, their best prediction accuracies, and the associated optimal predictors are shown in Figure 5.8.

In Figure 5.8, the states denoted with bold circles represent states in which the branch is predicted taken; the nonbold circles represent states that predict not taken. Similarly the bold edges represent state transitions when the branch is actually taken; the nonbold edges represent transitions corresponding to the branch

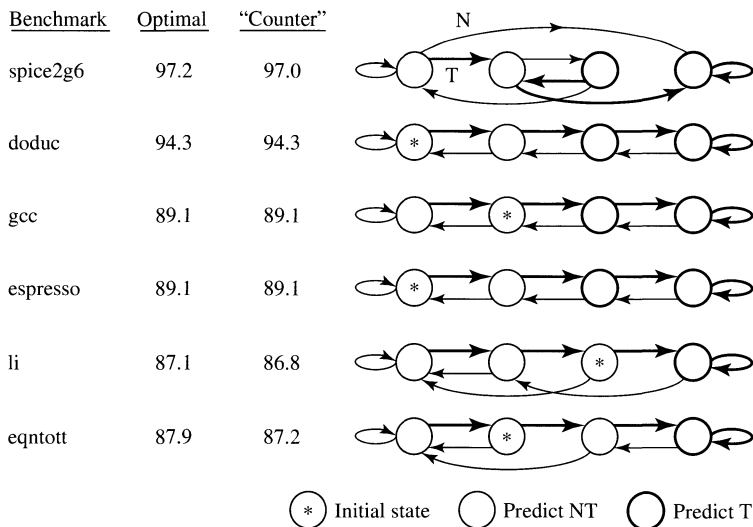


Figure 5.8

Optimal 2-Bit Branch Predictors for Six SPEC Benchmarks from the Nair Study.

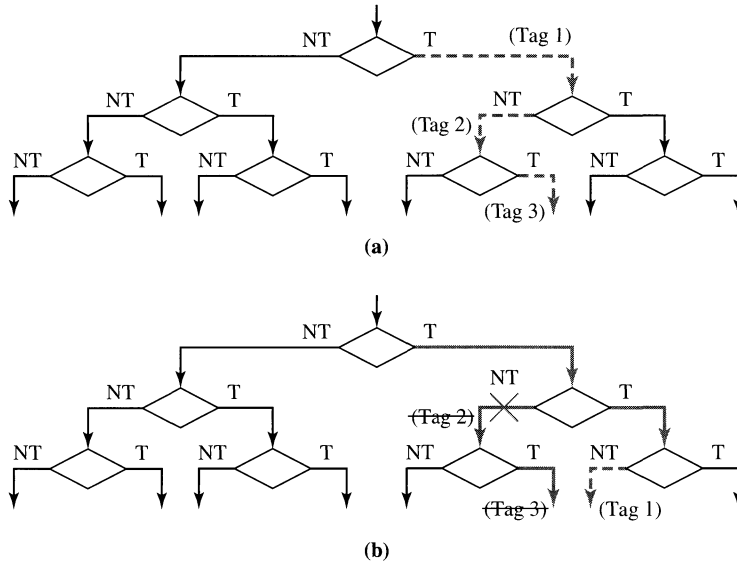
actually not taken. The state denoted with an asterisk indicates the initial state. The prediction accuracies for the optimal predictors of these six benchmarks range from 87.1% to 97.2%. Notice that the optimal predictors for *doduc*, *gcc*, and *espresso* are identical (disregarding the different initial state of the *gcc* predictor) and exhibit the behavior of a 2-bit up/down saturating counter. We can label the four states from left to right as 0, 1, 2, and 3, representing the four count values of a 2-bit counter. Whenever a branch is resolved taken, the count is incremented; and it is decremented otherwise. The two lower-count states predict a branch to be not taken, while the two higher-count states predict a branch to be taken. Figure 5.8 also provides the prediction accuracies for the six benchmarks if the 2-bit saturating counter predictor is used for all six benchmarks. The prediction accuracies for *spice2g6*, *li*, and *eqntott* only decrease minimally from their optimal values, indicating that the 2-bit saturating counter is a good candidate for general use on all benchmarks. In fact, the 2-bit saturating counter, originally invented by Jim Smith, has become a popular prediction algorithm in real and experimental designs.

The same study by Nair also investigated the effectiveness of counter-based predictors. With a 1-bit counter as the predictor, i.e., remembering the direction taken last time and predicting the same direction for the next time, the prediction accuracies ranged from 82.5% to 96.2%. As we have seen in Figure 5.8, a 2-bit counter yields an accuracy range of 86.8% to 97.0%. If a 3-bit counter is used, the increase in accuracy is minimal; accuracies range from 88.3% to 97.0%. Based on this study, the 2-bit saturating counter appears to be a very good choice for a history-based predictor. Direct-mapped branch history tables are assumed in this study. While some programs, such as *gcc*, have more than 7000 conditional branches, for most programs, the branch penalty due to aliasing in finite-sized branch history tables levels out at about 1024 entries for the table size.

5.1.4 Branch Misprediction Recovery

Branch prediction is a speculative technique. Any speculative technique requires mechanisms for validating the speculation. Dynamic branch prediction can be viewed as consisting of two interacting engines. The leading engine performs speculation in the front-end stages of the pipeline, while a trailing engine performs validation in the later stages of the pipeline. In the case of misprediction the trailing engine also performs recovery. These two aspects of branch prediction are illustrated in Figure 5.9.

Branch speculation involves predicting the direction of a branch and then proceeding to fetch along the predicted path of control flow. While fetching from the predicted path, additional branch instructions may be encountered. Prediction of these additional branches can be similarly performed, potentially resulting in speculating past multiple conditional branches before the first speculated branch is resolved. Figure 5.9(a) illustrates speculating past three branches with the first and the third branches being predicted taken and the second one predicted not taken. When this occurs, instructions from three speculative basic blocks are now resident in the machine and must be appropriately identified. Instructions from each speculative basic block are given the same identifying tag. In the example of

**Figure 5.9**

Two Aspects of Branch Prediction: (a) Branch Speculation; (b) Branch Validation/Recovery.

Figure 5.9(a), three distinct tags are used to identify the instructions from the three speculative basic blocks. A tagged instruction indicates that it is a speculative instruction, and the value of the tag identifies which basic block it belongs to. As a speculative instruction advances down the pipeline stages, the tag is also carried along. When speculating, the instruction addresses of all the speculated branch instructions (or the next sequential instructions) must be buffered in the event that recovery is required.

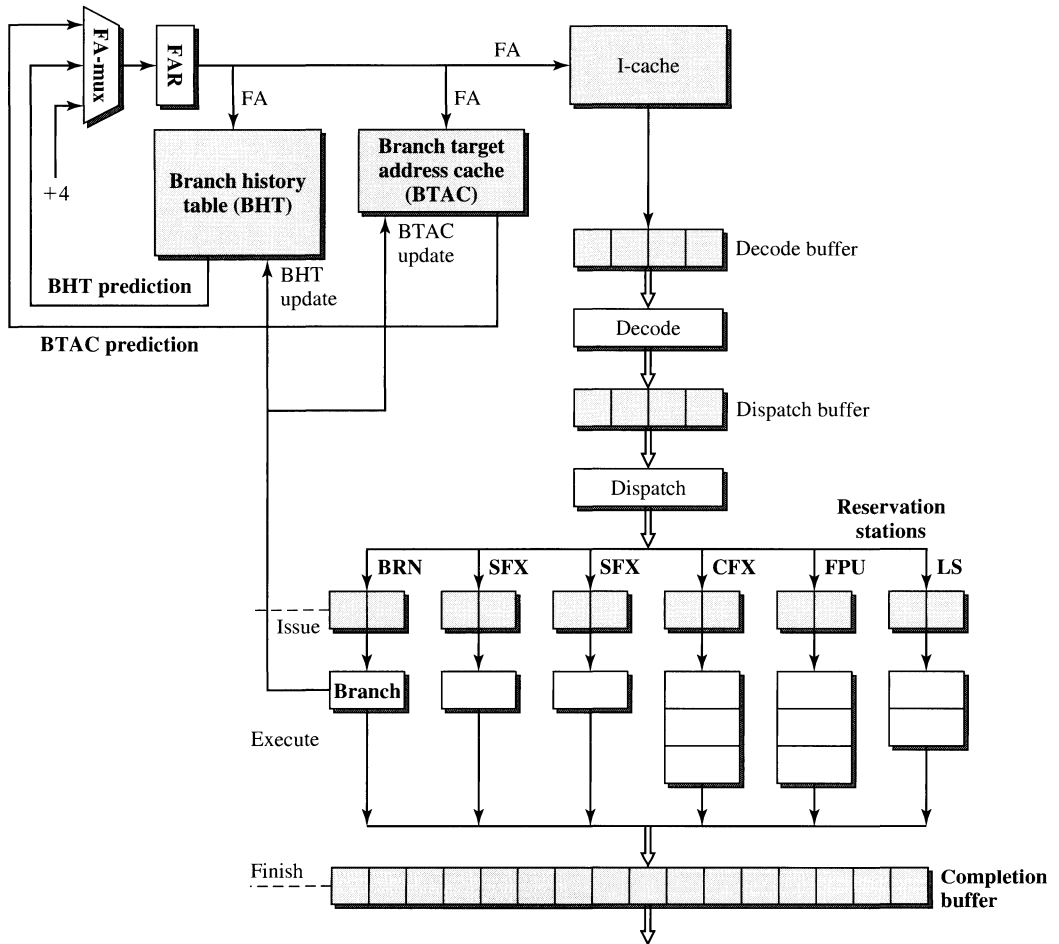
Branch validation occurs when the branch is executed and the actual direction of a branch is resolved. The correctness of the earlier prediction can then be determined. If the prediction turns out to be correct, the speculation tag is deallocated and all the instructions associated with that tag become nonspeculative and are allowed to complete. If a misprediction is detected, two actions are required; namely, the incorrect path must be terminated, and fetching from a new correct path must be initiated. To initiate a new path, the PC must be updated with a new instruction fetch address. If the incorrect prediction was a not-taken prediction, then the PC is updated with the computed branch target address. If the incorrect prediction was a taken prediction, then the PC is updated with the sequential (fall-through) instruction address, which is obtained from the previously buffered instruction address when the branch was predicted taken. Once the PC has been updated, fetching of instructions resumes along the new path, and branch prediction begins anew. To terminate the incorrect path, speculation tags are used. All the tags that are associated with the mispredicted branch are used to identify the

instructions that must be eliminated. All such instructions that are still in the decode and dispatch buffers as well as those in reservation station entries are invalidated. Reorder buffer entries occupied by these instructions are deallocated. Figure 5.9(b) illustrates this validation/recovery task when the second of the three predictions is incorrect. The first branch is correctly predicted, and therefore instructions with Tag 1 become nonspeculative and are allowed to complete. The second prediction is incorrect, and all the instructions with Tag 2 and Tag 3 must be invalidated and their reorder buffer entries must be deallocated. After fetching down the correct path, branch prediction can begin once again, and Tag 1 is used again to denote the instructions in the first speculative basic block. During branch validation, the associated BTB entry is also updated.

We now use the PowerPC 604 superscalar microprocessor to illustrate the implementation of dynamic branch prediction in a real superscalar processor. The PowerPC 604 is a four-wide superscalar capable of fetching, decoding, and dispatching up to four instructions in every machine cycle [IBM Corp., 1994]. Instead of a single unified BTB, the PowerPC 604 employs two separate buffers to support branch prediction, namely, the branch target address cache (BTAC) and the branch history table (BHT); see Figure 5.10. The BTAC is a 64-entry fully associative cache that stores the branch target addresses, while the BHT, a 512-entry direct-mapped table, stores the history bits of branches. The reason for this separation will become clear shortly.

Both the BTAC and the BHT are accessed during the fetch stage using the current instruction fetch address in the PC. The BTAC responds in one cycle; however, the BHT requires two cycles to complete its access. If a hit occurs in the BTAC, indicating the presence of a branch instruction in the current fetch group, a predict taken occurs and the branch target address retrieved from the BTAC is used in the next fetch cycle. Since the PowerPC 604 fetches four instructions in a fetch cycle, there can be multiple branches in the fetch group. Hence, the BTAC entry indexed by the fetch address contains the branch target address of the first branch instruction in the fetch group that is predicted to be taken. In the second cycle, or during the decode stage, the history bits retrieved from the BHT are used to generate a history-based prediction on the same branch. If this prediction agrees with the taken prediction made by the BTAC, the earlier prediction is allowed to stand. On the other hand, if the BHT prediction disagrees with the BTAC prediction, the BTAC prediction is annulled and fetching from the fall-through path, corresponding to predict not taken, is initiated. In essence, the BHT prediction can overrule the BTAC prediction. As expected, in most cases the two predictions agree. In some cases, the BHT corrects the wrong prediction made by the BTAC. It is possible, however, for the BHT to erroneously change the correct prediction of the BTAC; this occurs very infrequently. When a branch is resolved, the BHT is updated; and based on its updated content the BHT in turn updates the BTAC by either leaving an entry in the BTAC if it is to be predicted taken the next time, or deleting an entry from the BTAC if that branch is to be predicted not taken the next time.

The PowerPC 604 has four entries in the reservation station that feeds the branch execution unit. Hence, it can speculate past up to four branches; i.e., there

**Figure 5.10**

Branch Prediction in the PowerPC 604 Superscalar Microprocessor.

can be a maximum of four speculative branches present in the machine. To denote the four speculative basic blocks involved, a 2-bit tag is used to identify all speculative instructions. After a branch resolves, branch validation takes place and all speculative instructions either are made nonspeculative or are invalidated via the use of the 2-bit tag. Reorder buffer entries occupied by misspeculated instructions are deallocated. Again, this is performed using the 2-bit tag.

5.1.5 Advanced Branch Prediction Techniques

The dynamic branch prediction schemes discussed thus far have a number of limitations. Prediction for a branch is made based on the limited history of only that particular static branch instruction. The actual prediction algorithm does not take

into account the dynamic context within which the branch is being executed. For example, it does not make use of any information on the particular control flow path taken in arriving at that branch. Furthermore the same fixed algorithm is used to make the prediction regardless of the dynamic context. It has been observed experimentally that the behavior of certain branches is strongly correlated with the behavior of other branches that precede them during execution. Consequently more accurate branch prediction can be achieved with algorithms that take into account the branch history of other correlated branches and that can adapt the prediction algorithm to the dynamic branching context.

In 1991, Yeh and Patt proposed a two-level adaptive branch prediction technique that can potentially achieve better than 95% prediction accuracy by having a highly flexible prediction algorithm that can adapt to changing dynamic contexts [Yeh and Patt, 1991]. In previous schemes, a single branch history table is used and indexed by the branch address. For each branch address there is only one relevant entry in the branch history table. In the two-level adaptive scheme, a set of history tables is used. These are identified as the pattern history table (PHT); see Figure 5.11. Each branch address indexes to a set of relevant entries; one of these entries is then selected based on the dynamic branching context. The context is determined by a specific pattern of recently executed branches stored in a branch history shift register (BHSR); see Figure 5.11. The content of the BHSR is used to index into the PHT to select one of the relevant entries. The content of this entry is then used as the state for the prediction algorithm FSM to produce a prediction. When a branch is resolved, the branch result is used to update both the BHSR and the selected entry in the PHT.

The two-level adaptive branch prediction technique actually specifies a framework within which many possible designs can be implemented. There are two options

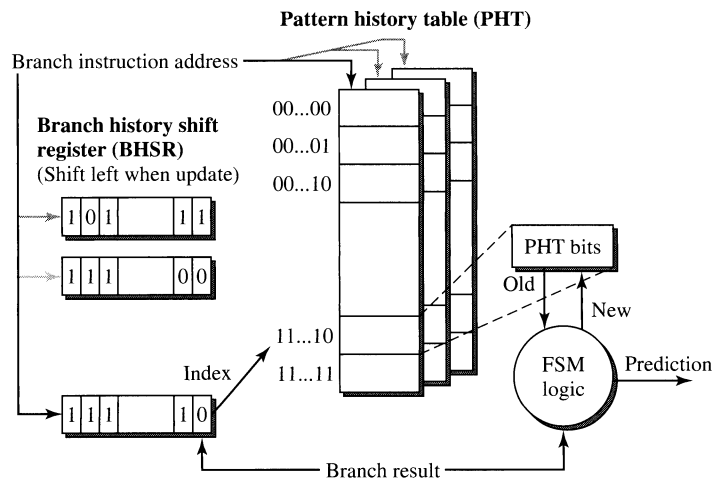


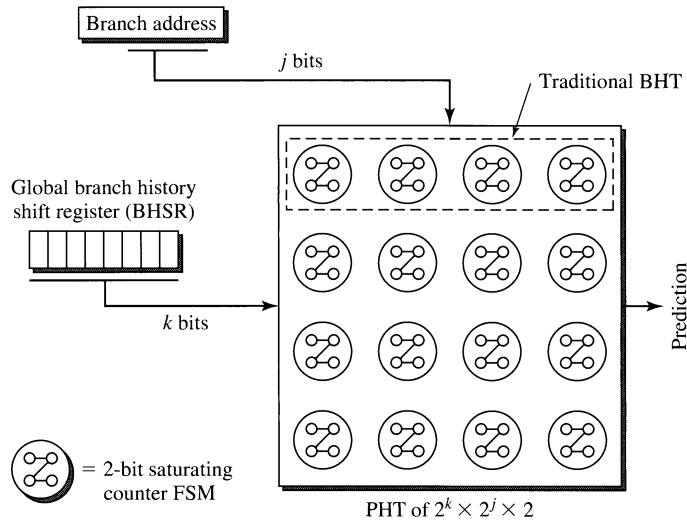
Figure 5.11
Two-Level Adaptive Branch Prediction of Yeh and Patt.
Source: Yeh and Patt, 1991.

to implementing the BHSR: *global* (G) and *individual* (P). The global implementation employs a single BHSR of k bits that tracks the branch directions of the last k dynamic branch instructions in program execution. These can involve any number (1 to k) of static branch instructions. The individual (called *per-branch* by Yeh and Patt) implementation employs a set of k -bit BHSRs as illustrated in Figure 5.11, one of which is selected based on the branch address. Essentially the global BHSR is shared by all static branches, whereas with individual BHSRs each BHSR is dedicated to each static branch or a subset of static branches if there is address aliasing when indexing into the set of BHSRs using the branch address. There are three options to implementing the PHT: *global* (g), *individual* (p), or *shared* (s). The global PHT uses a single table to support the prediction of all static branches. Alternatively, individual PHTs can be used in which each PHT is dedicated to each static branch (p) or a small subset of static branches (s) if there is address aliasing when indexing into the set of PHTs using the branch address. A third dimension to this design space involves the implementation of the actual prediction algorithm. When a history-based FSM is used to implement the prediction algorithm, Yeh and Patt identified such schemes as *adaptive* (A).

All possible implementations of the two-level adaptive branch prediction can be classified based on these three dimensions of design parameters. A given implementation can then be denoted using a three-letter notation; e.g., GAs represents a design that employs a single global BHSR, an adaptive prediction algorithm, and a set of PHTs with each being shared by a number of static branches. Yeh and Patt presented three specific implementations that are able to achieve a prediction accuracy of 97% for their given set of benchmarks:

- GAg: (1) BHSR of size 18 bits; (1) PHT of size $2^{18} \times 2$ bits.
- PAG: (512 $\times$ 4) BHSRs of size 12 bits; (1) PHT of size $2^{12} \times 2$ bits.
- PAs: (512 $\times$ 4) BHSRs of size 6 bits; (512) PHTs of size $2^6 \times 2$ bits.

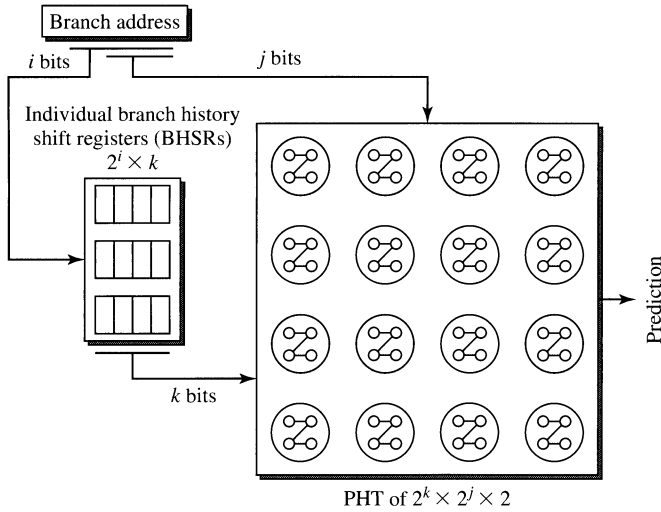
All three implementations use an adaptive (A) predictor that is a 2-bit FSM. The first implementation employs a global BHSR (G) of 18 bits and a global PHT (g) with 2^{18} entries indexed by the BHSR bits. The second implementation employs 512 sets (four-way set-associative) of 12-bit BHSRs (P) and a global PHT (g) with 2^{12} entries. The third implementation also employs 512 sets of four-way set-associative BHSRs (P), but each is only 6 bits wide. It also uses 512 PHTs (s), each having 2^6 entries indexed by the BHSR bits. Both the 512 sets of BHSRs and the 512 PHTs are indexed using 9 bits of the branch address. Additional branch address bits are used for the set-associative access of the BHSRs. The 512 PHTs are direct-mapped, and there can be aliasing, i.e., multiple branch addresses sharing the same PHT. From experimental data, such aliasing had minimal impact on degrading the prediction accuracy. Achieving greater than 95% prediction accuracy by the two-level adaptive branch prediction schemes is quite impressive; the best traditional prediction techniques can only achieve about 90% prediction accuracy. The two-level adaptive branch prediction approach has been adopted by a number of real designs, including the Intel Pentium Pro and the AMD/NexGen Nx686.

**Figure 5.12**

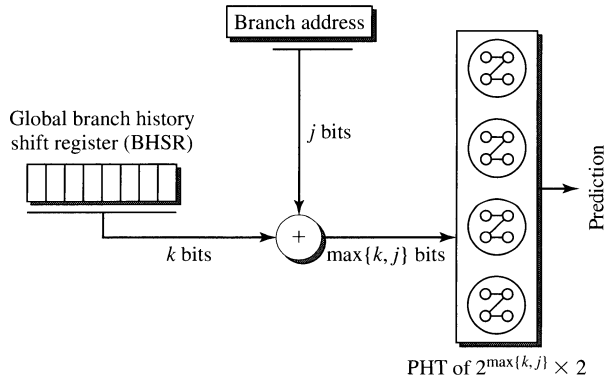
Correlated Branch Predictor with Global BHSR and Shared PHTs (GAs).

Following the original Yeh and Patt proposal, other studies by McFarling [1993], Young and Smith [1994], and Gloy et al. [1995] have gained further insights into two-level adaptive, or more recently called *correlated*, branch predictors. Figure 5.12 illustrates a correlated branch predictor with a global BHSR (G) and a shared PHT (s). The 2-bit saturating counter is used as the predictor FSM. The global BHSR tracks the directions of the last k dynamic branches and captures the dynamic control flow context. The PHT can be viewed as a single table containing a two-dimensional array, with 2^j columns and 2^k rows, of 2-bit predictors. If the branch address has n bits, a subset of j bits is used to index into the PHT to select one of the 2^j columns. Since j is less than n , some aliasing can occur where two different branch addresses can index into the same column of the PHT. Hence the designation of *shared* PHT. The k bits from the BHSR are used to select one of the 2^k entries in the selected column. The 2 history bits in the selected entry are used to make a history-based prediction. The traditional branch history table is equivalent to having only one row of the PHT that is indexed only by the j bits of the branch address, as illustrated in Figure 5.12 by the dashed rectangular block of 2-bit predictors in the first row of the PHT.

Figure 5.13 illustrates a correlated branch predictor with individual, or per-branch, BHSRs (P) and the same shared PHT (s). Similar to the GAs scheme, the PAs scheme also uses j bits of the branch address to select one of the 2^j columns of the PHT. However, i bits of the branch address, which can overlap with the j bits used to access the PHT, are used to index into a set of BHSRs. Depending on the branch address, one of the 2^i BHSRs is selected. Hence, each BHSR is associated with one particular branch address, or a set of branch addresses if there is aliasing. Essentially, instead of using a single BHSR to provide the dynamic control flow context for all static branches, multiple BHSRs are used to provide distinct dynamic control flow contexts for different subsets of static branches. This adds

**Figure 5.13**

Correlated Branch Predictor with Individual BHSRs and Shared PHTs (PAs).

**Figure 5.14**

The *gshare* Correlated Branch Predictor of McFarling.

Source: McFarling, 1993.

flexibility in tracking and exploiting correlations between different branch instructions. Each BHSR tracks the directions of the last k dynamic branches belonging to the same subset of static branches. Both the GAs and the PAs schemes require a PHT of size $2^k \times 2^j \times 2$ bits. The GAs scheme has only one k -bit BHSR whereas the PAs scheme requires 2^i k -bit BHSRs.

A fairly efficient correlated branch predictor called *gshare* was proposed by Scott McFarling [1993]. In this scheme, j bits from the branch address are “hashed” (via bitwise XOR function) with the k bits from a global BHSR; see Figure 5.14. The resultant $\max\{k, j\}$ bits are used to index into a PHT of size $2^{\max\{k, j\}} \times 2$ bits to

select one of the $2^{\max\{k,j\}}$ 2-bit branch predictors. The *gshare* scheme requires only one k -bit BHSR and a much smaller PHT, yet achieves comparable prediction accuracy to other correlated branch predictors. This scheme is used in the DEC Alpha 21264 4-way superscalar microprocessor [Keller, 1996].

5.1.6 Other Instruction Flow Techniques

The primary objective for instruction flow techniques is to supply as many useful instructions as possible to the execution core of the processor in every machine cycle. The two major challenges deal with conditional branches and taken branches. For a wide superscalar processor, to provide adequate conditional branch throughput, the processor must very accurately predict the outcomes and targets of multiple conditional branches in every machine cycle. For example, in a fetch group of four instructions, it is possible that all four instructions are conditional branches. Ideally one would like to use the addresses of all four instructions to index into a four-ported BTB to retrieve the history bits and target addresses of all four branches. A complex predictor can then make an overall prediction based on all the history bits. Speculative fetching can then proceed based on this prediction. Techniques for predicting multiple branches in every cycle have been proposed by Conte et al. [1995] as well as Rotenberg et al. [1996]. It is also important to ensure high accuracy in such predictions. Global branch history can be used in conjunction with per-branch history to achieve very accurate predictions. For those branches or sequences of branches that do not exhibit strongly biased branching behavior and therefore are not predictable, *dynamic eager execution* (DEE) has been proposed by Gus Uht [Uht and Sindagi, 1995]. DEE employs multiple PCs to simultaneously fetch from multiple addresses. Essentially the fetch stage pursues down multiple control flow paths until some branches are resolved, at which time some of the wrong paths are dynamically pruned by invalidating the instructions on those paths.

Taken branches are the second major obstacle to supplying enough useful instructions to the execution core. In a wide machine the fetch unit must be able to correctly process more than one taken branch per cycle, which involves predicting each branch's direction and target, and fetching, aligning, and merging instructions from multiple branch targets. An effective approach in alleviating this problem involves the use of a *trace cache* that was initially proposed by Eric Rotenberg [Rotenberg et al., 1996]. Since then, a form of trace caching has been implemented in Intel's most recent Pentium 4 superscalar microprocessor. Trace cache is a history-based fetch mechanism that stores dynamic instruction traces in a cache indexed by the fetch address and branch outcomes. These traces are assembled dynamically based on the dynamic branching behavior and can contain multiple nonconsecutive basic blocks. Whenever the fetch address hits in the trace cache, instructions are fetched from the trace cache rather than the instruction cache. Since a dynamic sequence of instructions in the trace cache can contain multiple taken branches but is stored sequentially, there is no need to fetch from multiple targets and no need for a multiported instruction cache or complex merging and aligning logic in the fetch stage. The trace cache can be viewed as doing dynamic

basic block reordering according to the dominant execution paths taken by a program. The merging and aligning can be done at completion time, when nonconsecutive basic blocks on a dominant path are first executed, to assemble a trace, which is then stored in one line of the trace cache. The goal is that once the trace cache is warmed up, most of the fetching will come from the trace cache instead of the instruction cache. Since the reordered basic blocks in the trace cache better match the dynamic execution order, there will be fewer fetches from nonconsecutive locations in the trace cache, and there will be an effective increase in the overall throughput of taken branches.

5.2 Register Data Flow Techniques

Register data flow techniques concern the effective execution of ALU (or register-register) type instructions in the execution core of the processor. ALU instructions can be viewed as performing the “real” work specified by the program, with control flow and load/store instructions playing the supportive roles of providing the necessary instructions and the required data, respectively. In the most ideal machine, branch and load/store instructions, being “overhead” instructions, should take no time to execute and the computation latency should be strictly determined by the processing of ALU instructions. The effective processing of these instructions is foundational to achieving high performance.

Assuming a load/store architecture, ALU instructions specify operations to be performed on source operands stored in registers. Typically an ALU instruction specifies a binary operation, two source registers where operands are to be retrieved, and a destination register where the result is to be placed. $R_i \leftarrow F_n(R_j, R_k)$ specifies a typical ALU instruction, the execution of which requires the availability of (1) F_n , the functional unit; (2) R_j and R_k , the two source operand registers; and (3) R_i , the destination register. If the functional unit F_n is not available, then a structural dependence exists that can result in a structural hazard. If one or both of the source operands in R_j and R_k are not available, then a hazard due to true data dependence can occur. If the destination register R_i is not available, then a hazard due to anti- and output dependences can occur.

5.2.1 Register Reuse and False Data Dependences

The occurrence of anti- and output dependences, or false data dependences, is due to the reuse of registers. If registers are never reused to store operands, then such false data dependences will not occur. The reuse of registers is commonly referred to as *register recycling*. Register recycling occurs in two different forms, one static and the other dynamic. The static form is due to optimization performed by the compiler and is presented first. In a typical compiler, toward the back end of the compilation process two tasks are performed: *code generation* and *register allocation*. The code generation task is responsible for the actual emitting of machine instructions. Typically the code generator assumes the availability of an unlimited number of symbolic registers in which it stores all the temporary data. Each symbolic register is used to store one value and is only written once, producing what is

commonly referred to as *single-assignment* code. However, an ISA has a limited number of architected registers, and hence the register allocation tool is used to map the unlimited number of symbolic registers to the limited and fixed number of architected registers. The register allocator attempts to keep as many of the temporary values in registers as possible to avoid having to move the data out to memory locations and reloading them later on. It accomplishes this by reusing registers. A register is written with a new value when the old value stored there is no longer needed; effectively each register is recycled to hold multiple values.

Writing of a register is referred to as the *definition* of a register and the reading of a register as the *use* of a register. After each definition, there can be one or more uses of that definition. The duration between the definition and the last use of a value is referred to as the *live range* of that value. After the last use of a live range, that register can be assigned to store another value and begin another live range. Register allocation procedures attempt to map nonoverlapping live ranges into the same architected register and maximize register reuse. In single-assignment code there is a one-to-one correspondence between symbolic registers and values. After register allocation, each architected register can receive multiple assignments, and the register becomes a variable that can take on multiple values. Consequently the one-to-one correspondence between registers and values is lost.

If the instructions are executed sequentially and a redefinition is never allowed to precede the previous definition or the last use of the previous definition, then the live ranges that share the same register will never overlap during execution and the recycling of registers does not induce any problem. Effectively, the one-to-one correspondence between values and registers can be maintained implicitly if all the instructions are processed in the original program order. However, in a superscalar machine, especially with out-of-order processing of instructions, register reading and writing operations can occur in an order different from the program order. Consequently the one-to-one correspondence between values and registers can potentially be perturbed; in order to ensure semantic correctness all anti- and output dependences must be detected and enforced. Out-of-order reading (writing) of registers can be permitted as long as all the anti- (output) dependences are enforced.

The dynamic form of register recycling occurs when a loop of instructions is repeatedly executed. With an aggressive superscalar machine capable of supporting many instructions in flight and a relatively small loop body being executed, multiple iterations of the loop can be simultaneously in flight in a machine. Hence, multiple copies of a register defining instruction from the multiple iterations can be simultaneously present in the machine, inducing the dynamic form of register recycling. Consequently anti- and output dependences can be induced among these dynamic instructions from the multiple iterations of a loop and must be detected and enforced to ensure semantic correctness of program execution.

One way to enforce anti- and output dependences is to simply stall the dependent instruction until the leading instruction has finished accessing the dependent register. If an anti- [write-after-read (WAR)] dependence exists between a pair of instructions, the trailing instruction (register updating instruction) must be stalled until the leading instruction has read the dependent register. If an output [write-after-write (WAW)] dependence exists between a pair of instructions, the trailing

instruction (register updating instruction) must be stalled until the leading instruction has first updated the register. Such stalling of anti- and output dependent instructions can lead to significant performance loss and is not necessary. Recall that such false data dependences are induced by the recycling of the architected registers and are not intrinsic to the program semantics.

5.2.2 Register Renaming Techniques

A more aggressive way to deal with false data dependences is to dynamically assign different *names* to the multiple definitions of an architected register and, as a result, eliminate the presence of such false dependences. This is called *register renaming* and requires the use of hardware mechanisms at run time to undo the effects of register recycling by reproducing the one-to-one correspondence between registers and values for all the instructions that might be simultaneously in flight. By performing register renaming, single assignment is effectively recovered for the instructions that are in flight, and no anti- and output dependences can exist among these instructions. This will allow the instructions that originally had false dependences between them to be executed in parallel.

A common way to implement register renaming is to use a separate rename register file (RRF) in addition to the architected register file (ARF). A straightforward way to implement the RRF is to simply duplicate the ARF and use the RRF as a shadow version of the ARF. This will allow each architected register to be renamed once. However, this is not a very efficient way to use the registers in the RRF. Many existing designs implement an RRF with fewer entries than the ARF and allow each of the registers in the RRF to be flexibly used to rename any one of the architected registers. This facilitates the efficient use of the rename registers, but does require a mapping table to store the pointers to the entries in the RRF. The use of a separate RRF in conjunction with a mapping table to perform renaming of the ARF is illustrated in Figure 5.15.

When a separate RRF is used for register renaming, there are implementation choices in terms of where to place the RRF. One option is to implement a separate stand-alone structure similar to the ARF and perhaps adjacent to the ARF. This is shown in Figure 5.15(a). An alternative is to incorporate the RRF as part of the reorder buffer, as shown in Figure 5.15(b). In both options a busy field is added to the ARF along with a mapping table. If the busy bit of a selected entry of the ARF is set, indicating the architected register has been renamed, the corresponding entry of the map table is accessed to obtain the tag or the pointer to an RRF entry. In the former option, the tag specifies a rename register and is used to index into the RRF; whereas in the latter option, the tag specifies a reorder buffer entry and is used to index into the reorder buffer.

Based on the diagrams in Figure 5.15, the difference between the two options may seem artificial; however, there are important subtle differences. If the RRF is incorporated as part of the reorder buffer, every entry of the reorder buffer contains an additional field that functions as a rename register, and hence there is a rename register allocated for every instruction in flight. This is a design based on worst-case scenario and may be wasteful since not every instruction defines a register. For example, branch instructions do not update any architected register. On the

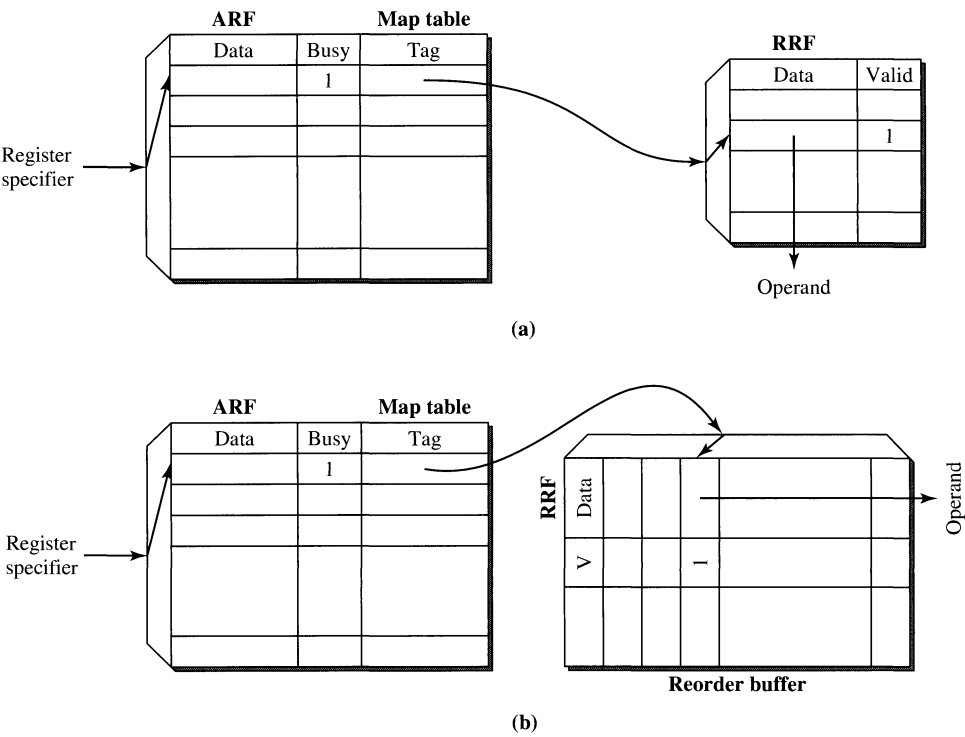
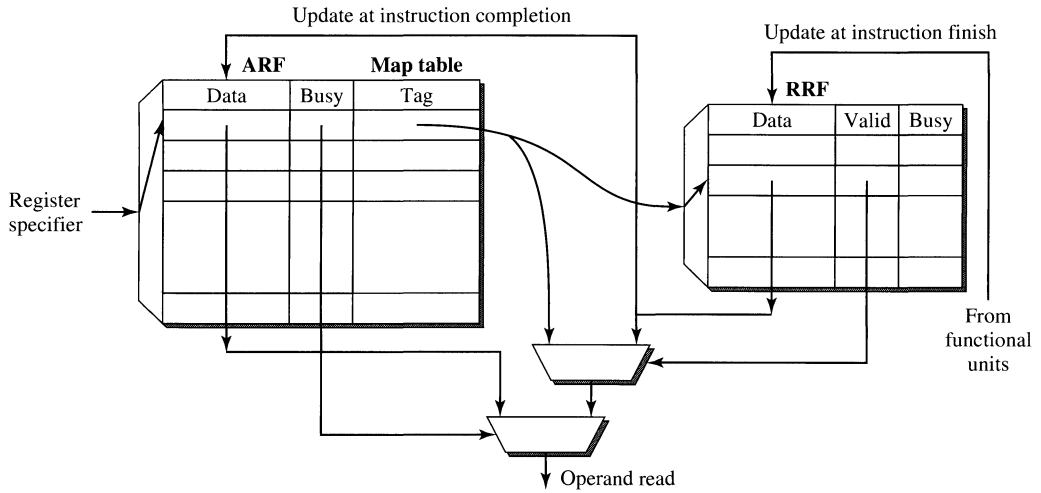


Figure 5.15
Rename Register File (RRF) Implementations: (a) Stand-Alone; (b) Attached to the Reorder Buffer.

other hand, a reorder buffer already contains ports to receive data from the functional units and to update the ARF at instruction completion time. When a separate stand-alone RRF is used, it introduces an additional structure that requires ports for receiving data from the functional units and for updating the ARF. The choice of which of the two options to implement involves design tradeoffs, and both options have been employed in real designs. We now focus on the stand-alone option to get a better feel of how register renaming actually works.

Register renaming involves three tasks: (1) *source read*, (2) *destination allocate*, and (3) *register update*. The first task of source read typically occurs during the decode (or possibly dispatch) stage and is for the purpose of fetching the register operands. When an instruction is decoded, its source register specifiers are used to index into a multiported ARF in order to fetch the register operands. Three possibilities can occur for each register operand fetch. First, if the busy bit is not set, indicating there is no pending write to the specified register and that the architected register contains the specified operand, the operand is fetched from the ARF. If the busy bit is set, indicating there is a pending write to that register and that the content of the architected register is stale, the corresponding entry of the map table is accessed to retrieve the rename tag. This rename tag specifies a

**Figure 5.16**

Register Renaming Tasks: Source Read, Destination Allocate, and Register Update.

rename register and is used to index into the RRF. Two possibilities can occur when indexing into the RRF. If the valid bit of the indexed entry is set, it indicates that the register-updating instruction has already finished execution, although it is still waiting to be completed. In this case, the source operand is available in the rename register and is retrieved from the indexed RRF entry. If the valid bit is not set, it indicates that the register-updating instruction still has not been executed and that the rename register has a pending update. In this case the tag, or the rename register specifier, from the map table is forwarded to the reservation station instead of to the source operand. This tag will be used later by the reservation station to obtain the operand when it becomes available. These three possibilities for source read are shown in Figure 5.16.

The task of destination allocate also occurs during the decode (or possibly dispatch) stage and has three subtasks, namely, *set busy bit*, *assign tag*, and *update map table*. When an instruction is decoded, its destination register specifier is used to index into the ARF. The selected architected register now has a pending write, and its busy bit must be set. The specified destination register must be mapped to a rename register. A particular unused (indicated by the busy bit) rename register must be selected. The busy bit of the selected RRF entry must be set, and the index of the selected RRF entry is used as a tag. This tag must then be written into the corresponding entry in the map table, to be used by subsequent dependent instructions for fetching their source operands.

While the task of register update takes place in the back end of the machine and is not part of the actual renaming activity of the decode/dispatch stage, it does have a direct impact on the operation of the RRF. Register update can occur in two separate steps; see Figure 5.16. When a register-updating instruction finishes execution, its result is written into the entry of the RRF indicated by the tag. Later on when this

instruction is completed, its result is then copied from the RRF into the ARF. Hence, register update involves updating first an entry in the RRF and then an entry in the ARF. These two steps can occur in back-to-back cycles if the register-updating instruction is at the head of the reorder buffer, or they can be separated by many cycles if there are other unfinished instructions in the reorder buffer ahead of this instruction. Once a rename register is copied to its corresponding architected register, its busy bit is reset and it can be used again to rename another architected register.

So far we have assumed that register renaming implementation requires the use of two separate physical register files, namely the ARF and the RRF. However, this assumption is not necessary. The architected registers and the rename registers can be pooled together and implemented as a single physical register file with its number of entries equal to the sum of the ARF and RRF entry counts. Such a pooled register file does not rigidly designate some of the registers as architected registers and others as rename registers. Each physical register can be flexibly assigned to be an architected register or a rename register. Unlike a separate ARF and RRF implementation which must physically copy a result from the RRF to the ARF at instruction completion, the pooled register file only needs to change the designation of a register from being a rename register to an architected register. This will save the data transfer interconnect between the RRF and the ARF. The key disadvantage of the pooled register file is its hardware complexity. A secondary disadvantage is that at context swap time, when the machine state must be saved, the subset of registers constituting the architected state of the machine must be explicitly identified before state saving can begin.

The pooled register file approach is used in the floating-point unit of the original IBM RS/6000 design and is illustrated in Figure 5.17 [Grohoski, 1990; Oehler and Groves, 1990]. In this design, 40 physical registers are implemented for supporting an ISA that specifies 32 architected registers. A mapping table is implemented, based on whose content any subset of 32 of the 40 physical registers can be designated as the architected registers. The mapping table contains 32 entries indexed by the 5-bit architected register specifier. Each entry when indexed returns a 6-bit specifier indicating the physical register to which the architected register has been mapped.

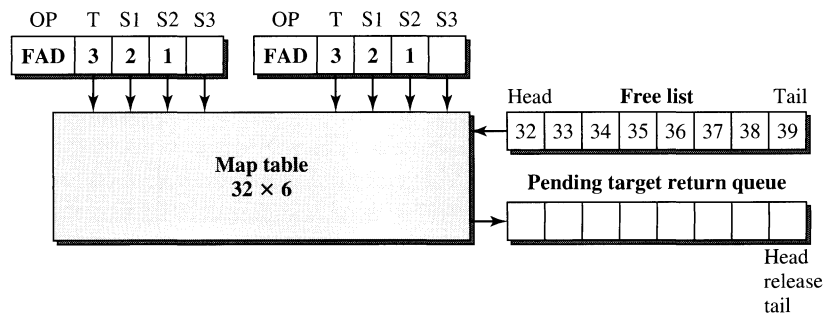


Figure 5.17
Floating-Point Unit (FPU) Register Renaming in the IBM RS/6000.

The floating-point unit (FPU) of the RS/6000 is a pipelined functional unit with the rename pipe stage preceding the decode pipe stage. The rename pipe stage contains the map table, two circular queues, and the associated control logic. The first queue is called the *free list* (FL) and contains physical registers that are available for new renaming. The second queue is called the *pending target return queue* (PTRQ) and contains those physical registers that have been used to rename architected registers that have been subsequently re-renamed in the map table. Physical registers in the PTRQ can be returned to the FL once the last use of that register has occurred. Two instructions can traverse the rename stage in every machine cycle. Because of the possibility of fused multiply-add (FMA) instructions that have three sources and one destination, each of the two instructions can contain up to four register specifiers. Hence, the map table must be eight-ported to support the simultaneous translation of the eight architected register specifiers. The map table is initialized with the identity mapping; i.e., architected register i is mapped to physical register i for $i = 0, 1, \dots, 31$. At initialization, physical registers 32 to 39 are placed in the FL and the PTRQ is empty.

When an instruction traverses the rename stage, its architected register specifiers are used to index into the map table to obtain their translated physical register specifiers. The eight-ported map table has 32 entries, indexed by the 5-bit architected register specifier, with each entry containing 6 bits indicating the physical register to which the architected register is mapped. The content of the map table represents the latest mapping of architected registers to physical registers and specifies the subset of physical registers that currently represents the architected registers.

In the FPU of the RS/6000, by design only load instructions can trigger a new renaming. Such register renaming prevents the FPU from stalling while waiting for loads to execute in order to enforce anti- and output dependences. When a load instruction traverses the rename stage, its destination register specifier is used to index into the map table. The current content of that entry of the map table is pushed out to the PTRQ, and the next physical register in the FL is loaded into the map table. This effectively renames the redefinition of that destination register to a different physical register. All subsequent instructions that specify this architected register as a source operand will receive the new physical register specifier as the source register. Beyond the rename stage, i.e., in the decode and execute stages, the FPU uses only physical register specifiers, and all true register dependences are enforced using the physical register specifiers.

The map table approach represents the most aggressive and versatile implementation of register renaming. Every physical register can be used to represent any redefinition of any architected register. There is significant hardware complexity required to implement the multiported map table and the logic to control the two circular queues. The return of a register in the PTRQ to the FL is especially troublesome due to the difficulty in identifying the last-use instruction of a register. However, unlike approaches based on the use of separate rename registers, at instruction completion time no copying of the content of the rename registers to the architected registers is necessary. On the other hand, when interrupts occur and as part of context swap, the subset of physical registers that constitute the current architected machine state must be explicitly determined based on the map table contents.

Most contemporary superscalar microprocessors implement some form of register renaming to avoid having to stall for anti- and output register data dependences induced by the reuse of registers. Typically register renaming occurs during the instruction decoding time, and its implementation can become quite complex, especially for wide superscalar machines in which many register specifiers for multiple instructions must be simultaneously renamed. It's possible that multiple redefinitions of a register can occur within a fetch group. Implementing a register renaming mechanism for wide superscalars without seriously impacting machine cycle time is a real challenge. To achieve high performance the serialization constraints imposed by false register data dependences must be eliminated; hence, dynamic register renaming is absolutely essential.

5.2.3 True Data Dependences and the Data Flow Limit

A RAW dependence between two instructions is called a true data dependence due to the producer-consumer relationship between these two instructions. The trailing consumer instruction cannot obtain its source operand until the leading producer instruction produces its result. A true data dependence imposes a serialization constraint between the two dependent instructions; the leading instruction must finish execution before the trailing instruction can begin execution. Such true data dependences result from the semantics of the program and are usually represented by a data flow graph or data dependence graph (DDG).

Figure 5.18 illustrates a code fragment for a fast Fourier transform (FFT) implementation. Two source-level statements are compiled into 16 assembly instructions, including load and store instructions. The floating-point array variables

```
w[i+k].ip = z[i].rp + z[m+i].rp;
w[i+j].rp = e[k+1].rp * (z[i].rp - z[m+i].rp) - e[k+1].ip * (z[i].ip - z[m+i].ip);
```

(a)

```
i1: f2 ← load,4(r2)
i2: f0 ← load,4(r5)
i3: f0 ← fadd,f2,f0
i4: 4(r6) ← store,f0
i5: f14 ← load,8(r7)
i6: f6 ← load,0(r2)
i7: f5 ← load,0(r3)
i8: f5 ← fsub,f6,f5
i9: f4 ← fmul,f14,f5
i10: f15 ← load,12(r7)
i11: f7 ← load,4(r2)
i12: f8 ← load,4(r3)
i13: f8 ← fsub,f7,f8
i14: f8 ← fmul,f15,f8
i15: f8 ← fsub,f4,f8
i16: 0(r8) ← store,f8
```

(b)

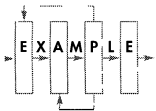
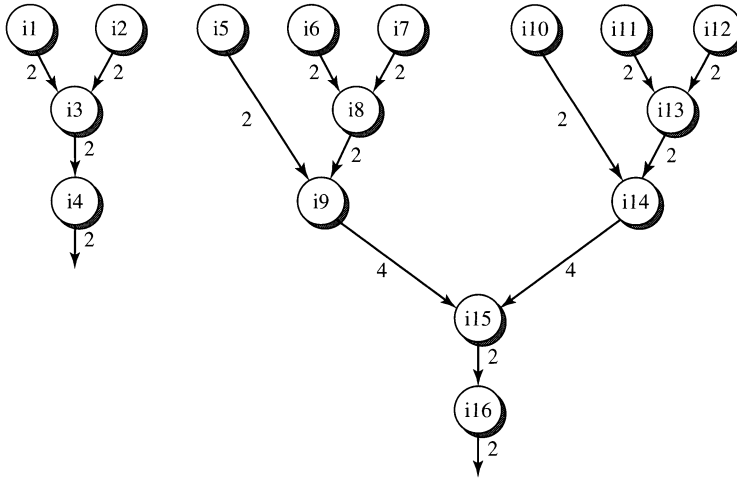


Figure 5.18

FFT Code Fragment: (a) Original Source Statements; (b) Compiled Assembly Instructions.

**Figure 5.19**

Data Flow Graph of the Code Fragment in Figure 5.18(b).

are stored in memory and must be first loaded before operations can be performed. After the computation, the results are stored back out to memory. Integer registers (*ri*) are used to hold addresses of arrays. Floating-point registers (*ff*) are used to hold temporary data. The DFG induced by the writing and reading of floating-point registers by the 16 instructions of Figure 5.18(b) is shown in Figure 5.19.

Each node in Figure 5.19 represents an instruction in Figure 5.18(b). A directed edge exists between two instructions if there exists a true data dependence between the two instructions. A dependent register can be identified for each of the dependence edges in the DFG. A latency can also be associated with each dependence edge. In Figure 5.19, each edge is labeled with the execution latency of the producer instruction. In this example, load, store, addition, and subtraction instructions are assumed to have two-cycle execution latency, while multiplication instructions require four cycles.

The latencies associated with dependence edges are cumulative. The longest dependence chain, measured in terms of total cumulative latency, is identified as the critical path of a DFG. Even assuming unlimited machine resources, a code fragment cannot be executed any faster than the length of its critical path. This is commonly referred to as the *data flow limit* to program execution and represents the best performance that can possibly be achieved. For the code fragment of Figure 5.19 the data flow limit is 12 cycles. The data flow limit is dictated by the true data dependences in the program. Traditionally, the *data flow execution model* stipulates that every instruction in a program begin execution immediately in the cycle following when all its operands become available. In effect, all existing register data flow techniques are attempts to approach the data flow limit.

5.2.4 The Classic Tomasulo Algorithm

The design of the IBM 360/91’s floating-point unit, incorporating what has come to be known as *Tomasulo’s algorithm*, laid the groundwork for modern superscalar processor designs [Tomasulo, 1967]. Key attributes of most contemporary register data flow techniques can be found in the classic Tomasulo algorithm, which deserves an in-depth examination. We first introduce the original design of the floating-point unit of the IBM 360, and then describe in detail the modified design of the FPU in the IBM 360/91 that incorporated Tomasulo’s algorithm, and finally illustrate its operation and effectiveness in processing an example code sequence.

The original design of the IBM 360 floating-point unit is shown in Figure 5.20. The FPU contains two functional units: one floating-point add unit and one floating-point multiply/divide unit. There are three register files in the FPU: the floating-point registers (FLRs), the floating-point buffers (FLBs), and the store data buffers (SDBs). There are four FLR registers; these are the architected floating-point registers. Floating-point instructions with storage-register or storage-storage addressing modes are preprocessed. Address generation and memory

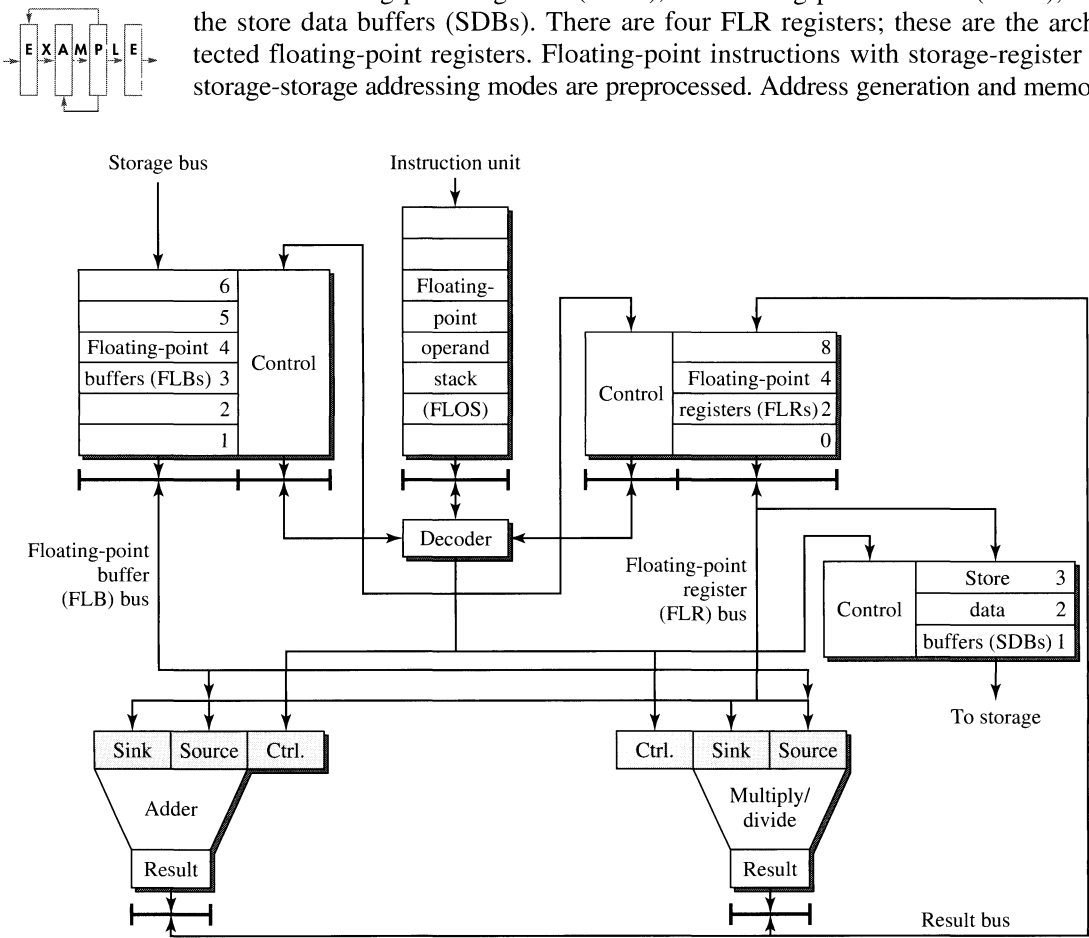


Figure 5.20
The Original Design of the IBM 360 Floating-Point Unit.

accessing are performed outside of the FPU. When the data are retrieved from the memory, they are loaded into one of the six FLB registers. Similarly if the destination of an instruction is a memory location, the result to be stored is placed in one of the three SDB registers and a separate unit accesses the SDBs to complete the storing of the result to a memory location. Using these two additional register files, the FLBs, and the SDBs, to support storage-register and storage-storage instructions, the FPU effectively functions as a register-register machine.

In the IBM 360/91, the instruction unit (IU) decodes all the instructions and passes all floating-point instructions (in order) to the floating-point operation stack (FLOS). In the FPU, floating-point instructions are then further decoded and issued in order from the FLOS to the two functional units. The two functional units are not pipelined and incur multiple-cycle latencies. The adder incurs 2 cycles for add instructions, while the multiply/divide unit incurs 3 cycles and 12 cycles for performing multiply and divide instructions, respectively.

In the mid-1960s, IBM began developing what eventually became Model 91 of the Systems 360 family. One of the goals was to achieve concurrent execution of multiple floating-point instructions and to sustain a throughput of one instruction per cycle in the instruction pipeline. This is quite aggressive considering the complex addressing modes of the 360 ISA and the multicycle latencies of the execution units. The end result is a modified FPU in the 360/91 that incorporated Tomasulo's algorithm; see Figure 5.21.

Tomasulo's algorithm consists of adding three new mechanisms to the original FPU design, namely, reservation stations, the common data bus, and register tags. In the original design, each functional unit has a single buffer on its input side to hold the instruction currently being executed. If a functional unit is busy, issuing of instructions by FLOS will stall whenever the next instruction to be issued requires the same functional unit. To alleviate this structural bottleneck, multiple buffers, called *reservation stations*, are attached to the input side of each functional unit. The adder unit has three reservation stations, while the multiply/divide unit has two. These reservation stations are viewed as virtual functional units; as long as there is a free reservation station, the FLOS can issue an instruction to that functional unit even if it is currently busy executing another instruction. Since the FLOS issues instructions in order, this will prevent unnecessary stalling due to unfortunate ordering of different floating-point instruction types.

With the availability of reservation stations, instructions can also be issued to the functional units by the FLOS even though not all their operands are yet available. These instructions can wait in the reservation station for their operands and only begin execution when they become available. The *common data bus* (CDB) connects the outputs of the two functional units to the reservation stations as well as the FLRs and SDB registers. Results produced by the functional units are broadcast into the CDB. Those instructions in the reservation stations needing the results as their operands will latch in the data from the CDB. Those registers in the FLR and SDB that are the destinations of these results also latch in the same data from the CDB. The CDB facilitates the forwarding of results directly from producer instructions to consumer instructions waiting in the reservation stations

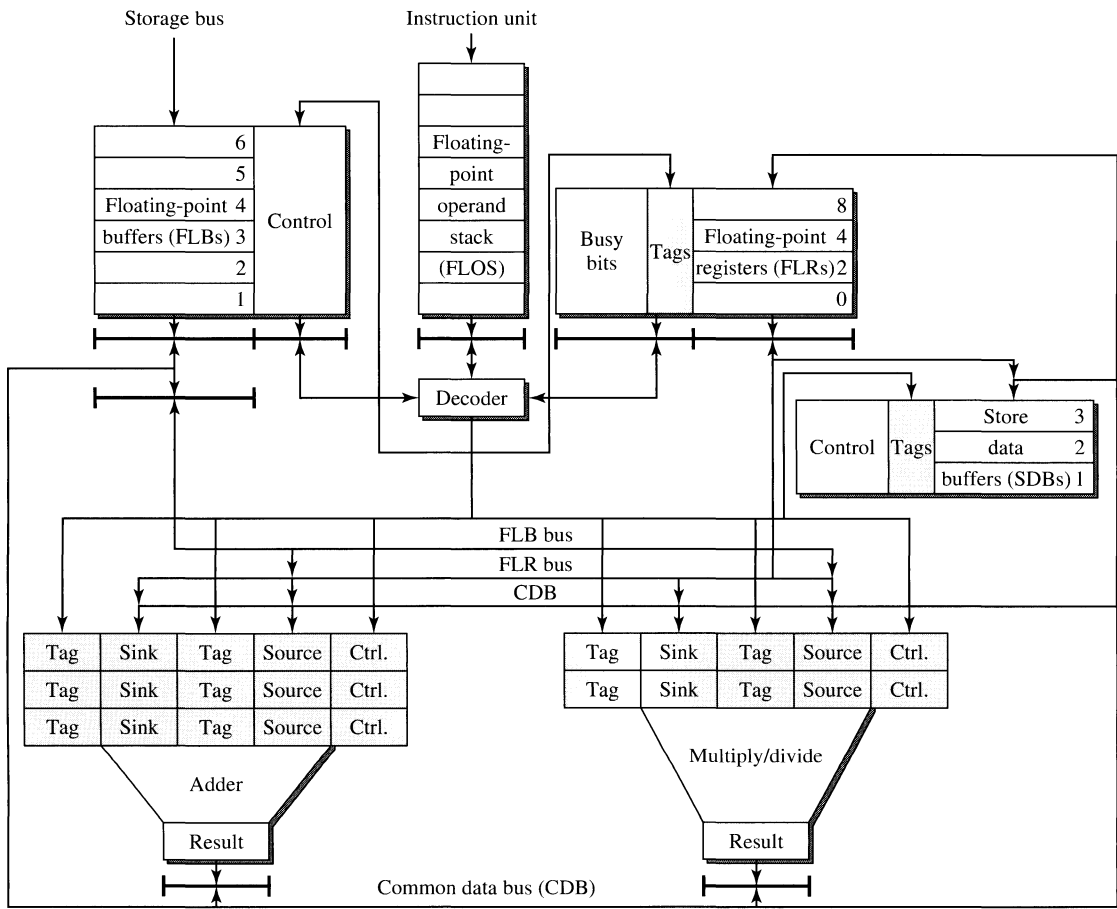
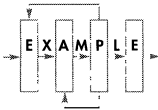


Figure 5.21

The Modified Design of the IBM 360/91 Floating-Point Unit with Tomasulo's Algorithm.



without having to go through the registers. Destination registers are updated simultaneously with the forwarding of results to dependent instructions. If an operand is coming from a memory location, it will be loaded into a FLB register once memory accessing is performed. Hence, the FLB can also output onto the CDB, allowing a waiting instruction in a reservation station to latch in its operand. Consequently, the two functional units and the FLBs can drive data onto the CDB, and the reservation station's FLRs and SDBs can latch in data from the CDB.

When the FLOS is dispatching an instruction to a functional unit, it allocates a reservation station and checks to see if the needed operands are available. If an operand is available in the FLRs, then the content of that register in the FLRs is copied to the reservation station; otherwise a tag is copied to the reservation station instead. The tag indicates where the pending operand is going to come from.

The pending operand can come from a producer instruction currently resident in one of the five reservation stations, or it can come from one of the six FLB registers. To uniquely identify one of these 11 possible sources for a pending operand, a 4-bit tag is required. If one of the two operand fields of a reservation station contains a tag instead of the actual operand, it indicates that this instruction is waiting for a pending operand. When that pending operand becomes available, the producer of that operand drives the tag along with the actual operand onto the CDB.

A waiting instruction in a reservation station uses its tag to monitor the CDB. When it detects a tag match on the CDB, it then latches in the associated operand. Essentially the producer of an operand broadcasts the tag and the operand on the CDB; all consumers of that operand monitor the CDB for that tag, and when the broadcasted tag matches their tag, they then latch in the associated operand from the CDB. Hence, all possible destinations of pending operands must carry a tag field and must monitor the CDB for a tag match. Each reservation station contains two operand fields, each of which must carry a tag field since each of the two operands can be pending. The four FLRs and the three registers in the SDB must also carry tag fields. This is a total of 17 tag fields representing 17 places that can monitor and receive operands; see Figure 5.22. The tag field at each potential consumer site is used in an associative fashion to monitor for possible matching of its content with the tag value being broadcasted on the CDB. When a tag match occurs, the consumer latches in the broadcasted operand.

The IBM 360 floating-point instructions use a two-address instruction format. Two source operands can be specified. The first operand specifier is called the *sink* because it also doubles as the destination specifier. The second operand specifier is called the *source*. Each reservation station has two operand fields, one for the sink and the other for the source. Each operand field is accompanied by a tag field. If an operand field contains real data, then its tag field is set to zero. Otherwise, its tag field identifies the source where the pending operand will be coming from, and is used to monitor the CDB for the availability of the pending operand. Whenever

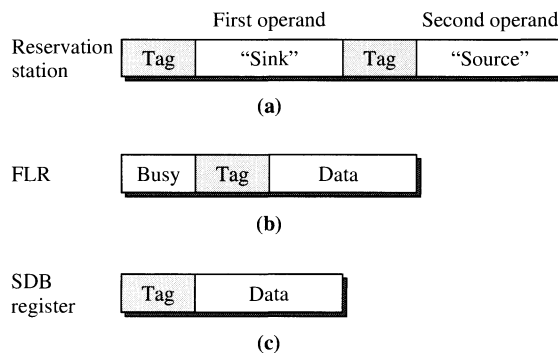


Figure 5.22

The Use of Tag Fields in (a) A Reservation Station, (b) A FLR, and (c) A SDB Register.

an instruction is dispatched by the FLOS to a reservation station, the data in the FLR corresponding to the sink operand are retrieved and copied to the reservation station. At the same time, the “busy” bit associated with this FLR is set, indicating that there is a pending update of that register, and the tag value that identifies the particular reservation station to which the instruction is being dispatched is written into the tag field of the same FLR. This clearly identifies which of the reservation stations will eventually produce the updated data for this FLR. Subsequently if a trailing instruction specifies this register as one of its source operands, when it is dispatched to a reservation station, only the tag field (called the *pseudo-operand*) will be copied to the corresponding tag field in the reservation station and not the actual data. When the busy bit is set, it indicates that the data in the FLR are stale and the tag represents the source from which the real data will come. Other than reservation stations and FLRs, SDB registers can also be destinations of pending operands and hence a tag field is required for each of the three SDB registers.

We now use an example sequence of instructions to illustrate the operation of Tomasulo’s algorithm. We deviate from the actual IBM 360/91 design in several ways to help clarify the example. First, instead of the two-address format of the IBM 360 instructions, we will use three-address instructions to avoid potential confusion. The example sequence contains only register-register instructions. To reduce the number of machine cycles we have to trace, we will allow the FLOS to dispatch (in program order) up to two instructions in every cycle. We also assume that an instruction can begin execution in the same cycle that it is dispatched to a reservation station. We keep the same latencies of two and three cycles for add and multiply instructions, respectively. However, we allow an instruction to forward its result to dependent instructions during its last execution cycle, and a dependent instruction can begin execution in the next cycle. The tag values of 1, 2, and 3 are used to identify the three reservation stations of the adder functional unit, while 4 and 5 are used to identify the two reservation stations of the multiply/divide functional unit. These tag values are called the *IDs* of the reservation stations. The example sequence consists of the following four register-register instructions.

w: $R4 \leftarrow R0 + R8$

x: $R2 \leftarrow R0 * R4$

y: $R4 \leftarrow R4 + R8$

z: $R8 \leftarrow R4 * R2$

Figure 5.23 illustrates the first three cycles of execution. In cycle 1, instructions w and x are dispatched (in order) to reservation stations 1 and 4. The destination registers of instructions w and x are R4 and R2 (i.e., FLRs 4 and 2), respectively. The busy bits of these two registers are set. Since instruction w is dispatched to reservation station 1, the tag value of 1 is entered into the tag field of R4, indicating that the instruction in reservation station 1 will produce the result for updating R4. Similarly the tag value of 4 is entered into the tag field of R2. Both source operands of instruction w are available, so it begins execution immediately. Instruction x

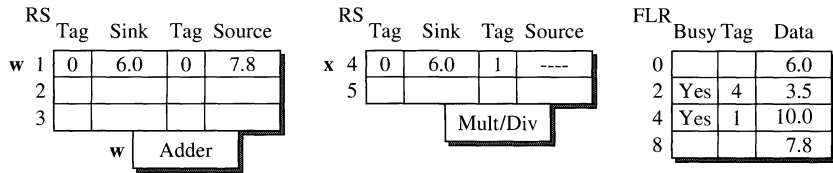
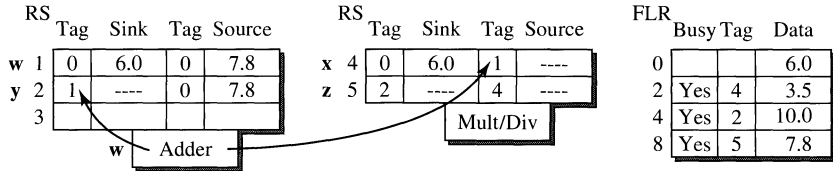
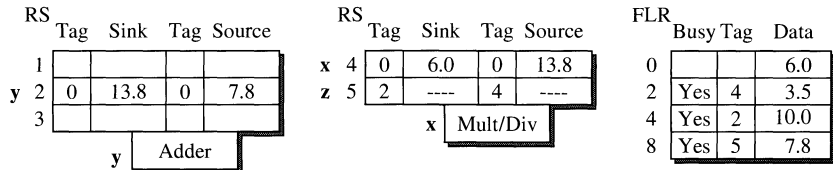
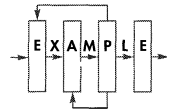
CYCLE 1 Dispatched instruction(s): **w, x** (in order)**CYCLE 2** Dispatched instruction(s): **y, z** (in order)**CYCLE 3** Dispatched instruction(s): _____**Figure 5.23**

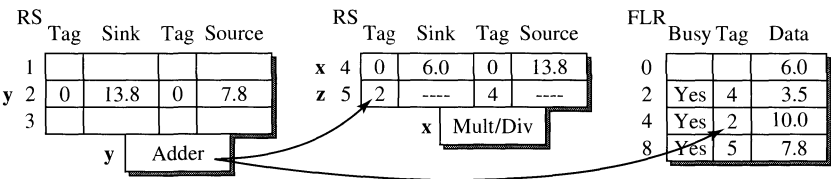
Illustration of Tomasulo's Algorithm on an Example Instruction Sequence (Part 1).



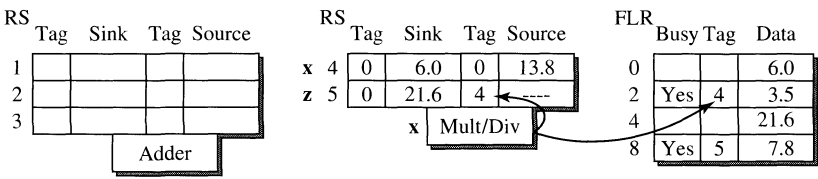
requires the result (R4) of instruction w for its second (source) operand. Hence when instruction x is dispatched to reservation station 4, the tag field of the second operand is written the tag value of 1, indicating that the instruction in reservation station 1 will produce the needed operand.

During cycle 2, instructions y and z are dispatched (in order) to reservation stations 2 and 5, respectively. Because it needs the result of instruction w for its first operand, instruction y, when it is dispatched to reservation station 2, receives the tag value of 1 in the tag field of the first operand. Similarly instruction z, dispatched to reservation station 5, receives the tag values of 2 and 4 in its two tag fields, indicating that reservation stations 2 and 4 will eventually produce the two operands it needs. Since R4 is the destination of instruction y, the tag field of R4 is updated with the new tag value of 2, indicating reservation station 2 (i.e., instruction y) is now responsible for the pending update of R4. The busy bit of R4 remains set. The busy bit of R8 is set when instruction z is dispatched to reservation station 5, and the tag field of R8 is set to 5. At the end of cycle 2, instruction w finishes execution and broadcasts its ID (reservation station 1) and its result onto the CDB. All the tag fields containing the tag value of 1 will trigger a tag match and latch in the broadcasted result. The first tag field of reservation station 2 (holding instruction y) and the second tag field of reservation station 4 (holding instruction x)

CYCLE 4 Dispatched instruction(s): _____



CYCLE 5 Dispatched instruction(s): _____



CYCLE 6 Dispatched instruction(s): _____

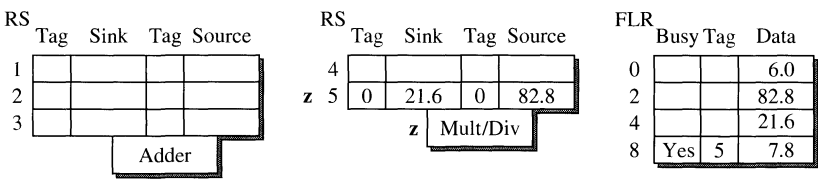
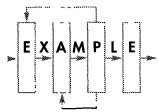


Figure 5.24

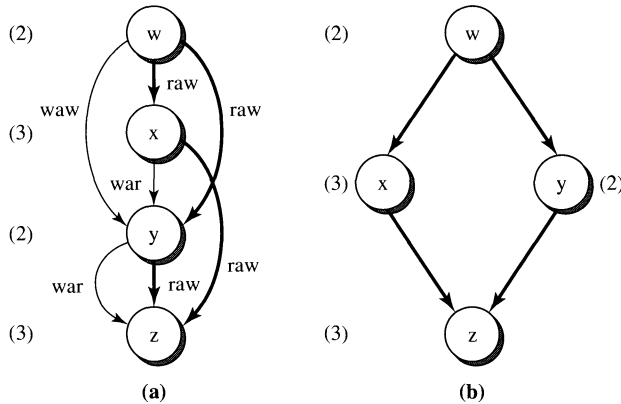
Illustration of Tomasulo's Algorithm on an Example Instruction Sequence (Part 2).



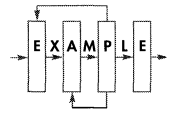
have such tag matches. Hence the result of instruction w is forwarded to dependent instructions x and y.

In cycle 3, instruction y begins execution in the adder unit, and instruction x begins execution in the multiply/divide unit. Instruction y finishes execution in cycle 4 (see Figure 5.24) and broadcasts its result on the CDB along with the tag value of 2 (its reservation station ID). The first tag field in reservation station 5 (holding instruction z) and the tag field of R4 have tag matches and pull in the result of instruction y. Instruction x finishes execution in cycle 5 and broadcasts its result on the CDB along with the tag value of 4. The second tag field in reservation station 5 (holding instruction z) and the tag field of R2 have tag matches and pull in the result of instruction x. In cycle 6, instruction z begins execution and finishes in cycle 8.

Figure 5.25(a) illustrates the data flow graph of this example sequence of four instructions. The four solid arcs represent the four true data dependences, while the other three arcs represent the anti- and output dependences. Instructions are dispatched in program order. Anti-dependences are resolved by copying an operand at dispatch time to the reservation station. Hence, it is not possible for a trailing instruction to overwrite a register before an earlier instruction has a chance to read that register. If the operand is still pending, the dispatched instruction will receive

**Figure 5.25**

Data Flow Graphs of the Example Instruction Sequence: (a) All Data Dependencies; (b) True Data Dependencies.



the tag for that operand. When that operand becomes available, the instruction will receive that operand via a tag match in its reservation station.

As an instruction is dispatched, the tag field of its destination register is written with the reservation station ID of that instruction. When a subsequent instruction with the same destination register is dispatched, the same tag field will be updated with the reservation station ID of this new instruction. The tag field of a register always contains the reservation station ID of the latest updating instruction. If there are multiple instructions in flight that have the same destination register, only the latest instruction will be able to update that register. Output dependences are implicitly resolved by making it impossible for an earlier instruction to update a register after a later instruction has updated the same register. This does introduce the problem of not being able to support precise exception since the register file does not necessarily evolve through all its sequential states; i.e., a register can potentially miss an intermediate update. For example, in Figure 5.23, at the end of cycle 2, instruction w should have updated its destination register R4. However, instruction y has the same destination register, and when it was dispatched earlier in that cycle, the tag field of R4 was changed from 1 to 2 anticipating the update of R4 by instruction y. At the end of cycle 2 when instruction w broadcasts its tag value of 1, the tag field of R4 fails to trigger a tag match and does not pull in the result of instruction w. Eventually R4 will be updated by instruction y. However, if an exception is triggered by instruction x, precise exception will be impossible since the register file does not evolve through all its sequential states.

Tomasulo's algorithm resolves anti- and output dependences via a form of register renaming. Each definition of an FLR triggers the renaming of that register to a register tag. This tag is taken from the ID of the reservation station containing the instruction that redefines that register. This effectively removes false dependences from causing pipeline stalls. Hence, the data flow limit is strictly determined by the true data dependences. Figure 5.25(b) depicts the data flow graph involving only

true data dependences. As shown in Figure 5.25(a) if all four instructions were required to execute sequentially to enforce all the data dependences, including anti- and output dependences, the total latency required for executing this sequence of instructions would be 10 cycles, given the latencies of 2 and 3 cycles for addition and multiplication instructions, respectively. When only true dependences are considered, Figure 5.25(b) reveals that the critical path is only 8 cycles, i.e., the path involving instructions w, x, and z. Hence, the data flow limit for this sequence of four instructions is 8 cycles. This limit is achieved by Tomasulo's algorithm as demonstrated in Figures 5.23 and 5.24.

5.2.5 Dynamic Execution Core

Most current state-of-the-art superscalar microprocessors consist of an out-of-order execution core sandwiched between an in-order front end, which fetches and dispatches instructions in program order, and an in-order back end, which completes and retires instructions also in program order. The out-of-order execution core (also referred to as the *dynamic execution core*), resembling a refinement of Tomasulo's algorithm, can be viewed as an embedded data flow, or *micro-dataflow*, engine that attempts to approach the data flow limit in instruction execution. The operation of such a dynamic execution core can be described according to the three phases in the pipeline, namely, *instruction dispatching*, *instruction execution*, and *instruction completion*; see Figure 5.26.

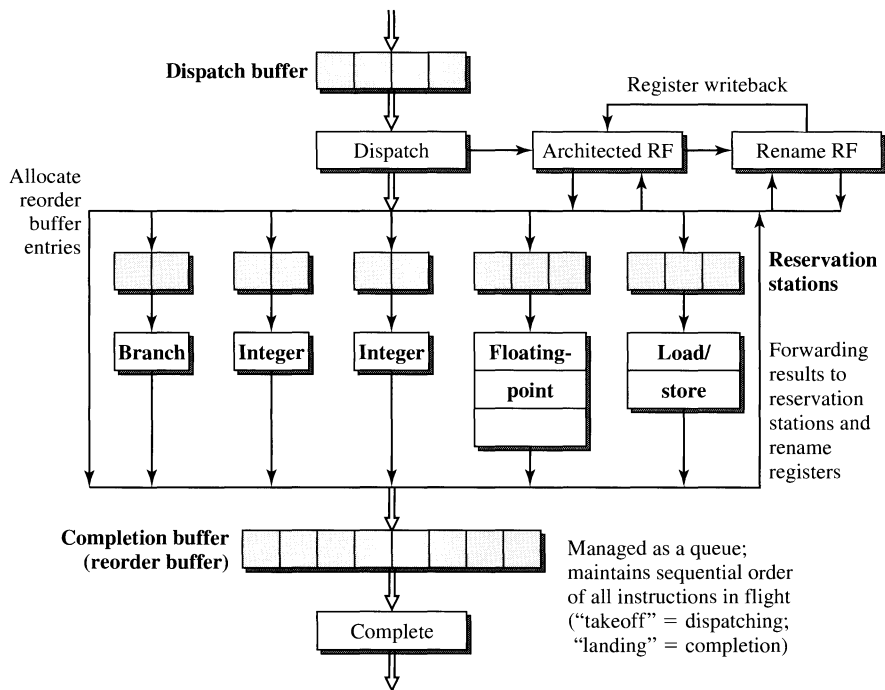


Figure 5.26

Micro-Dataflow Engine for Dynamic Execution.

The instruction dispatching phase consists of *renaming* of destination registers, *allocating* of reservation station and reorder buffer entries, and *advancing* instructions from the dispatch buffer to the reservation stations. For ease of presentation, we assume here that register renaming is performed in the dispatch stage. All redefinitions of architected registers are renamed to rename registers. Trailing uses of these redefinitions are assigned the corresponding rename register specifiers. This ensures that all producer-consumer relationships are properly identified and all false register dependences are removed.

Instructions in the dispatch buffer are then dispatched to the appropriate reservation stations based on instruction type. Here we assume the use of distributed reservation stations, and we use *reservation station* to refer to the (multientry) instruction buffer attached to each functional unit and *reservation station entry* to refer to one of the entries of this buffer. Simultaneous with the allocation of reservation station entries for the dispatched instructions is the allocation of entries in the reorder buffer for the same instructions. Reorder buffer entries are allocated according to program order.

Typically, for an instruction to be dispatched there must be the availability of a rename register, a reservation station entry, and a reorder buffer entry. If any one of these three is not available, instruction dispatching is stalled. The actual dispatching of instructions from the dispatch buffer entries to the reservation station entries is via a complex routing network. If the connectivity of this routing network is less than that of a full crossbar (this is frequently the case in real designs), then stalling can also occur due to resource contention in the routing network.

The instruction execution phase consists of *issuing* of ready instructions, *executing* the issued instructions, and *forwarding* of results. Each reservation station is responsible for identifying instructions that are ready to execute and for scheduling their execution. When an instruction is first dispatched to a reservation station, it may not have all its source operands and therefore must wait in the reservation station. Waiting instructions continually monitor the busses for tag matches. When a tag match is triggered, indicating the availability of the pending operand, the result being broadcasted is latched into the reservation station entry. When an instruction in a reservation station entry has all its operands, it becomes ready for execution and can be issued into the functional unit. In a given machine cycle if multiple instructions in a reservation station are ready, a scheduling algorithm is used (typically oldest first) to pick one of them for issuing into the functional unit to begin execution. If there is only one functional unit connected to a reservation station (as is the case for distributed reservation stations), then that reservation station can only issue one instruction per cycle.

Once issued into a functional unit, an instruction is executed. Functional units can vary in terms of their latency. Some have single-cycle latencies, others have fixed multiple-cycle latencies. Certain functional units can require a variable number of cycles, depending on the values of the operands and the operation being performed. Typically, even with function units that require multiple-cycle latencies, once an instruction begins execution in a pipelined functional unit, there is no further stalling of that instruction in the middle of the execution pipeline since all data dependences have been resolved prior to issuing and there shouldn't be any resource contention.

When an instruction finishes execution, it asserts its destination tag (i.e., the specifier of the rename register assigned for its destination) and the actual result onto a forwarding bus. All dependent instructions waiting in the reservation stations will trigger a tag match and latch in the broadcasted result. This is how an instruction forwards its result to other dependent instructions without requiring the intermediate steps of updating and then reading of the dependent register. Concurrent with result forwarding, the RRF uses the broadcasted tag as an index and loads the broadcasted result into the selected entry of the RRF.

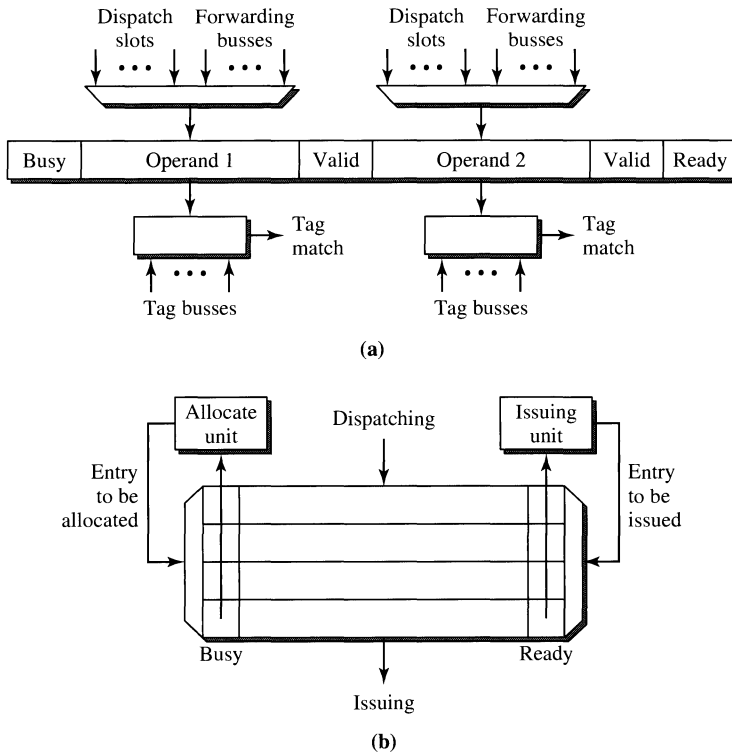
Typically a reservation station entry is deallocated when its instruction is issued in order to allow another trailing instruction to be dispatched into it. Reservation station saturation can cause instruction dispatch to stall. Certain instructions whose execution can induce an exceptional condition may require rescheduling for execution in a future cycle. Frequently, for these instructions, their reservation station entries are not deallocated until they finish execution without triggering any exceptions. For example, a load instruction can potentially trigger a D-cache miss that may require many cycles to service. Instead of stalling the functional unit, such an excepting load can be reissued from the reservation station after the miss has been serviced.

In a dynamic execution core as described previously, a producer-consumer relationship is satisfied without having to wait for the writing and then the reading of the dependent register. The dependent operand is directly forwarded from the producer instruction to the consumer instruction to minimize the latency incurred due to the true data dependence. Assuming that an instruction can be issued in the same cycle that it receives its last pending operand via a forwarding bus, if there is no other instruction contending for the same functional unit, then this instruction should be able to begin execution in the cycle immediately following the availability of all its operands. Hence, if there are adequate resources such that no stalling due to structural dependences occurs, then the dynamic execution core should be able to approach the data flow limit.

5.2.6 Reservation Stations and Reorder Buffer

Other than the functional units, the critical components of the dynamic execution core are the *reservation stations* and the *reorder buffer*. The operations of these components dictate the function of the dynamic execution core. Here we present the issues associated with the implementation of the reservation station and the reorder buffer. We present their organization and behavior with special focus on loading and unloading of an entry of a reservation station and the reorder buffer.

There are three tasks associated with the operation of a reservation station: *dispatching*, *waiting*, and *issuing*. A typical reservation station is shown in Figure 5.27(b), and the various fields in an entry of a reservation station are illustrated in Figure 5.27(a). Each entry has a busy bit, indicating that the entry has been allocated, and a ready bit, indicating that the instruction in that entry has all its source operands. Dispatching involves loading an instruction from the dispatch buffer into an entry of the reservation station. Typically the dispatching of an instruction requires the following three steps: select a free, i.e., not busy, reservation

**Figure 5.27**

Reservation Station Mechanisms: (a) A Reservation Station Entry;
 (b) Dispatching into and Issuing from a Reservation Station.

station entry; load operands and/or tags into the selected entry; and set the busy bit of that entry. The selection of a free entry is based on the busy bits and is performed by the allocate unit. The allocate unit examines all the busy bits and selects one of the nonbusy entries to be the allocated entry. This can be implemented using a priority encoder. Once an entry is allocated, the operands and/or tags of the instruction are loaded into the entry. Each entry has two operand fields, each of which has an associated valid bit. If the operand field contains the actual operand, then the valid bit is set. If the field contains a tag, indicating a pending operand, then its valid bit is reset and it must wait for the operand to be forwarded. Once an entry is allocated, its busy bit must be set.

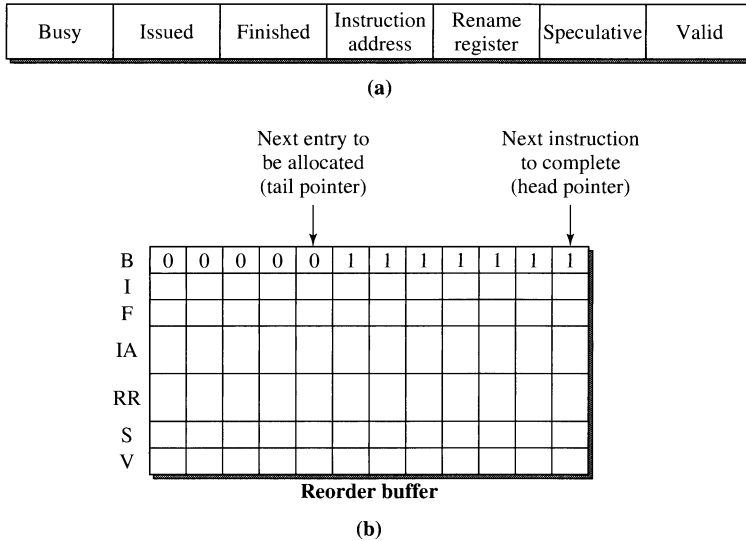
An instruction with a pending operand must wait in the reservation station. When a reservation station entry is waiting for a pending operand, it must continuously monitor the tag busses. When a tag match occurs, the operand field latches in the forwarded result and sets its valid bit. When both operand fields are valid, the ready bit is set, indicating that the instruction has all its source operands and is ready to be issued. This is usually referred to as *instruction wake up*.

The issuing step is responsible for selecting a ready instruction in the reservation station and issues it into the functional unit. This is usually referred to as *instruction select*. All the ready instructions are identified by their ready bits being set. The selecting of a ready instruction is performed by the issuing unit based on a scheduling heuristic; see Figure 5.27(b). The heuristic can be based on program order or how long each ready instruction has been waiting in the reservation station. Typically when an instruction is issued into the functional unit, its reservation station entry is deallocated by resetting the busy bit.

A large reservation station can be quite complex to implement. On its input side, it must support many possible sources, including all the dispatch slots and forwarding busses; see Figure 5.27(a). The data routing network on its input side can be quite complex. During the waiting step, all operand fields of a reservation station with pending operands must continuously compare their tags against potentially multiple tag busses. This is comparable to doing an associative search across all the reservation station entries involving multiple keys (tag busses). If the number of entries is small, this is quite feasible. However, as the number of entries increases, the increase in complexity is quite significant. This portion of the hardware is commonly referred to as the *wake-up logic*. When the entry count increases, it also complicates the issuing unit and the scheduling heuristic in selecting the best ready instruction to issue. This portion of the hardware is commonly referred to as the *select logic*. In any given machine cycle, there can be multiple ready instructions. The select logic must determine the best one to issue. For a superscalar machine, a reservation station can potentially support multiple instruction issues per cycle, in which case the select logic must pick the best subset of instructions to issue among all the ready instructions.

The reorder buffer contains all the instructions that are *in flight*, i.e., all the instructions that have been dispatched but not yet completed architecturally. These include all the instructions waiting in the reservation stations and executing in the functional units and those that have finished execution but are waiting to be completed in program order. The status of each instruction in the reorder buffer can be tracked using several bits in each entry of the reorder buffer. Each instruction can be in one of several states, i.e., waiting execution, in execution, and finished execution. These status bits are updated as an instruction traverses from one state to the next. An additional bit can also be used to indicate whether an instruction is speculative (in the predicted path) or not. If speculation can cross multiple branches, additional bits can be employed to identify which speculative basic block an instruction belongs to. When a branch is resolved, a speculative instruction can become nonspeculative (if the prediction is correct) or invalid (if the prediction is incorrect). Only finished and nonspeculative instructions can be completed. An instruction marked invalid is not architecturally completed when exiting the reorder buffer. Figure 5.28(a) illustrates the fields typically found in a reorder buffer entry; in this figure the rename register field is also included.

The reorder buffer is managed as a circular queue using a head pointer and a tail pointer; see Figure 5.28(b). The tail pointer is advanced when reorder buffer entries are allocated at instruction dispatch. The number of entries that can be

**Figure 5.28**

(a) Reorder Buffer Entry; (b) Reorder Buffer Organization.

allocated per cycle is limited by the dispatch bandwidth. Instructions are completed from the head of the queue. From the head of the queue as many instructions that have finished execution can be completed as the completion bandwidth allows. The completion bandwidth is determined by the capacity of another routing network and the ports available for register writeback. One of the critical issues is the number of write ports to the architected register file that are needed to support the transferring of data from the rename registers (or the reorder buffer entries if they are used as rename registers) to the architected registers. When an instruction is completed, its rename register and its reorder buffer entry are deallocated. The head pointer to the reorder buffer is also appropriately updated. In a way the reorder buffer can be viewed as the heart or the central control of the dynamic execution core because the status of all in-flight instructions is tracked by the reorder buffer.

It is possible to combine the reservation stations and the reorder buffer into one single structure, called the *instruction window*, that manages all the instructions in flight. Since at dispatch an entry in the reservation station and an entry in the reorder buffer must be allocated for each instruction, they can be combined as one entry in the instruction window. Hence, instructions are dispatched into the instruction window, entries of the instruction window monitor the tag busses for pending operands, results are forwarded into the instruction window, instructions are issued from the instruction window when ready, and instructions are completed from the instruction window. The size of the instruction window determines the maximum number of instructions that can be simultaneously in flight within the machine and consequently the degree of instruction-level parallelism that can be achieved by the machine.

5.2.7 Dynamic Instruction Scheduler

The dynamic instruction scheduler is the heart of a dynamic execution core. We use the term *dynamic instruction scheduler* to include the instruction window and its associated instruction wake-up and select logic. Currently there are two styles to the design of the dynamic instruction scheduler, namely, *with data capture* and *without data capture*.

Figure 5.29(a) illustrates a scheduler with data capture. With this style of scheduler design, when dispatching an instruction, those operands that are ready are copied from the register file (either architected or physical) into the instruction window; hence, we have the term *data captured*. For the operands that are not ready, tags are copied into the instruction window and used to latch in the operands when they are forwarded by the functional units. Results are forwarded to their waiting instructions in the instruction window. In effect, result forwarding and instruction wake up are combined, a la Tomasulo's algorithm. A separate forwarding path is needed to also update the register file so that subsequent dependent instructions can grab their source operands from the register file when they are being dispatched.

Some recent microprocessors have adopted a different style that does not employ data capture in the scheduler design; see Figure 5.29(b). In this style, register read is performed after the scheduler, as instructions are being issued to the functional units. At instruction dispatch there is no copying of operands into the instruction window; only tags (or pointers) for operands are loaded into the window. The scheduler still performs tag match to wake up ready instructions. However, results from functional units are only forwarded to the register file. All ready instructions that are issued obtain their operands directly from the register file just prior to execution. In effect, result forwarding and instruction wake up are decoupled. For instruction wake up only the tag needs to be forwarded to the scheduler. With the non-data-captured style of scheduler, the size (width) of the instruction window can be significantly reduced, and the much wider result-forwarding path to the scheduler is not needed.

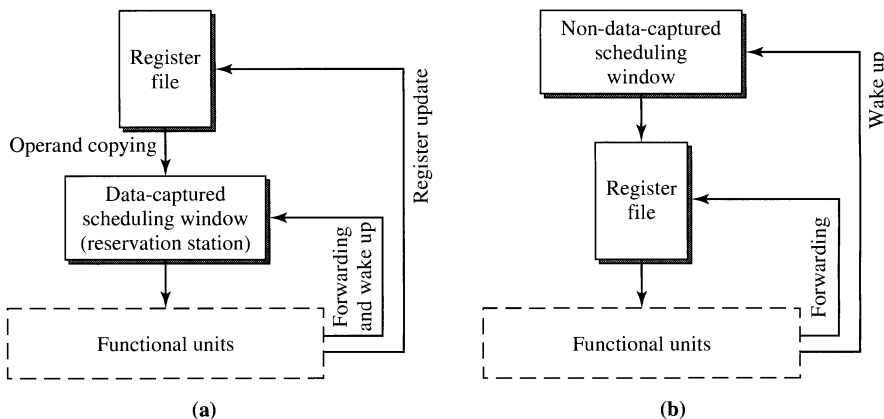


Figure 5.29

Dynamic Instruction Scheduler Design: (a) With Data Capture; (b) Without Data Capture.

There is a close relationship between register renaming and instruction scheduling. As stated earlier, one purpose for doing dynamic register renaming is to eliminate the false dependences induced by register recycling. Another purpose is to establish the producer-consumer relationship between two dependent instructions. A true data dependence is determined by the common rename register specifier in the producer and consumer instructions. The rename register specifier can function as the tag for result forwarding. In the non-data-captured scheduler of Figure 5.29(b) the register specifiers are used to access the register file for retrieving source operands; the (destination) register specifiers are used as tags for waking up dependent instructions in the scheduler. For the data-captured type of scheduler of Figure 5.29(a), the tags used for result forwarding and instruction wake up do not have to be actual register specifiers. The tags are mainly used to identify producer-consumer relationships between dependent instructions and can be assigned arbitrarily. For example, Tomasulo's algorithm uses reservation station IDs as the tags for forwarding results to dependent instructions as well as for updating architected registers. There is no explicit register renaming involving physical rename registers.

5.2.8 Other Register Data Flow Techniques

For many years the data flow limit has been assumed to be an absolute theoretical limit and the ultimate performance goal. Extensive research efforts on data flow architectures and data flow machines have been going on for over three decades. The data flow limit assumes that true data dependences are absolute and cannot possibly be overcome. Interestingly, in the late 1960s and the early 1970s a similar assumption was made concerning control dependences. It was generally thought that control dependences are absolute and that when encountering a conditional branch instruction there is no choice but to wait for that conditional branch to be executed before proceeding to the next instruction due to the uncertainty of the actual control flow. Since then, we have witnessed tremendous strides made in the area of branch prediction techniques. Conditional branches and associated control dependences are no longer absolute barriers and can frequently be overcome by speculating on the direction and the target address of the branch. What made such speculation possible is that frequently the outcome of a branch instruction is quite predictable. It wasn't until 1995 that researchers began to also question the absoluteness of true data dependences.

In 1996 several research papers appeared that proposed the concept of *value prediction*. The first paper by Lipasti, Wilkerson, and Shen focused on predicting load values based on the observation that frequently the values being loaded by a particular static load instruction are quite predictable [Lipasti et al., 1996]. Their second paper generalized the same basic idea for predicting the result of ALU instructions [Lipasti and Shen, 1996]. Experimental data based on real input data sets indicate that the results produced by many instructions are actually quite predictable. The notion of *value locality* indicates that certain instructions tend to repeatedly produce the same small set (sometimes one) of result values. By tracking the results produced by these instructions, future values can become predictable based on the historical values. Since these seminal papers, numerous papers have been published in recent years proposing various designs of value predictors [Mendelson and Gabbay, 1997; Sazeides and Smith, 1997; Calder et al., 1997; Gabbay and Mendelson, 1997;

1998a; 1998b; Calder et al., 1999]. In a recent study, it was shown that a hybrid value predictor can achieve prediction rates of up to 80% and a realistic design incorporating value prediction can achieve IPC improvements in the range of 8.6% to 23% for the SPEC benchmarks [Wang and Franklin, 1997].

When the result of an instruction is correctly predicted via value prediction, typically performed during the fetch stage, a subsequent dependent instruction can begin execution using this speculative result without having to wait for the actual decoding and execution of the leading instruction. This effectively removes the serialization constraint imposed by the true data dependence between these two instructions. In a way this particular dependence edge in the data flow graph is effectively removed when correct value prediction is performed. Hence, value prediction provides the potential to exceed the classical data flow limit. Of course, validation is still required to ensure that the prediction is correct and becomes the new limit on instruction execution throughput. Value prediction becomes effective in increasing machine performance if misprediction rarely occurs and the misprediction penalty is small (e.g., zero or one cycle) and if the validation latency is less than the average instruction execution latency. Clearly, efficient implementation of value prediction is crucial in ensuring its efficacy in improving performance.

Another recently proposed idea is called *dynamic instruction reuse* [Sodani and Sohi, 1997]. Similar to the concept of value locality, it has been observed through experiments with real programs that frequently the same sequence of instructions is repeatedly executed using the same set of input data. This results in redundant computation being performed by the machine. Dynamic instruction reuse techniques attempt to track such redundant computations, and when they are detected, the previous results are used without performing the redundant computations. These techniques are nonspeculative; hence, no validation is required. While value prediction can be viewed as the elimination of certain dependence edges in the data flow graph, dynamic instruction reuse techniques attempt to remove both nodes and edges of a subgraph from the data flow graph. A much earlier research effort had shown that such elimination of redundant computations can yield significant performance gains for programs written in functional languages [Harbison, 1980; 1982]. A more recent study also yields similar data on the presence of redundant computations in real programs [Richardson, 1992]. This is an area that is currently being actively researched, and new insightful results can be expected.

We will revisit these advanced register data flow techniques in Chapter 10 in greater detail.

5.3 Memory Data Flow Techniques

Memory instructions are responsible for moving data between the main memory and the register file, and they are essential for supporting the execution of ALU instructions. Register operands needed by ALU instructions must first be loaded from memory. With a limited number of registers, during the execution of a program not all the operands can be kept in the register file. The compiler generates *spill code* to temporarily place certain operands out to the main memory and to

reload them when they are needed. Such spill code is implemented using store and load instructions. Typically, the compiler only allocates scalar variables into registers. Complex data structures, such as arrays and linked lists, that far exceed the size of the register file are usually kept in the main memory. To perform operations on such data structures, load and store instructions are required. The effective processing of load/store instructions can minimize the overhead of moving data between the main memory and the register file.

The processing of load/store instructions and the resultant memory data flow can become a bottleneck to overall machine performance due to the potential long latency for executing memory instructions. The long latency of load/store instructions results from the need to compute a memory address and the need to access a memory location. To support virtual memory, the computed memory address (called the *virtual address*) also needs to be translated into a physical address before the physical memory can be accessed. Cache memories are very effective in reducing the effective latency for accessing the main memory. Furthermore, various techniques have been developed to reduce the overall latency and increase the overall throughput for processing load/store instructions.

5.3.1 Memory Accessing Instructions

The execution of memory data flow instructions occurs in three steps: memory address generation, memory address translation, and data memory accessing. We first state the basis for these three steps and then describe the processing of load/store instructions in a superscalar pipeline.

The register file and the main memory are defined by the instruction set architecture for data storage. The main memory as defined in an instruction set architecture is a collection of 2^n memory locations with random access capability; i.e., every memory location is identified by an n -bit address and can be directly accessed with the same latency. Just like the architected register file, the main memory is an architected entity and is visible to the software instructions. However, unlike the register file, the address that identifies a particular memory location is usually not explicitly stored as part of the instruction format. Instead, a memory address is usually generated based on a register and an offset specified in the instruction. Hence, address generation is required and involves the accessing of the specified register and the adding of the offset value.

In addition to address generation, address translation is required when virtual memory is implemented in a system. The architected main memory constitutes the virtual address space of the program and is viewed by each program as its private address space. The physical memory that is implemented in a machine constitutes the physical address space, which may be smaller than the virtual address space and may even be shared by multiple programs. Virtual memory is a mechanism that maps the virtual address space of a program to the physical address space of the machine. With such address mapping, virtual memory is able to support the execution of a program with a virtual address space that is larger than the physical address space, and the multiprogramming paradigm by mapping multiple virtual address spaces to the same physical address space. This mapping mechanism involves the translation of the

computed effective address, i.e., the virtual address, into a physical address that can be used to access the physical memory. This mechanism is usually implemented using a mapping table, and address translation is performed via a table lookup.

The third step in processing a load/store instruction is memory accessing. For load instructions data are read from a memory location and stored into a register, while for store instructions a register value is stored into a memory location. While the first two steps of address generation and address translation are performed in identical fashion for both loads and stores, the third step is performed differently for loads and stores by a superscalar pipeline.

In Figure 5.30, we illustrate these three steps as occurring in three pipeline stages. The first pipe stage performs effective address generation. We assume the typical addressing mode of register indirect with an offset for both load and store instructions. For a load instruction, as soon as the address register operand is available, it is issued into the pipelined functional unit and the effective address is generated by the first pipe stage. A store instruction must wait for the availability of both the address register and the data register operands before it is issued.

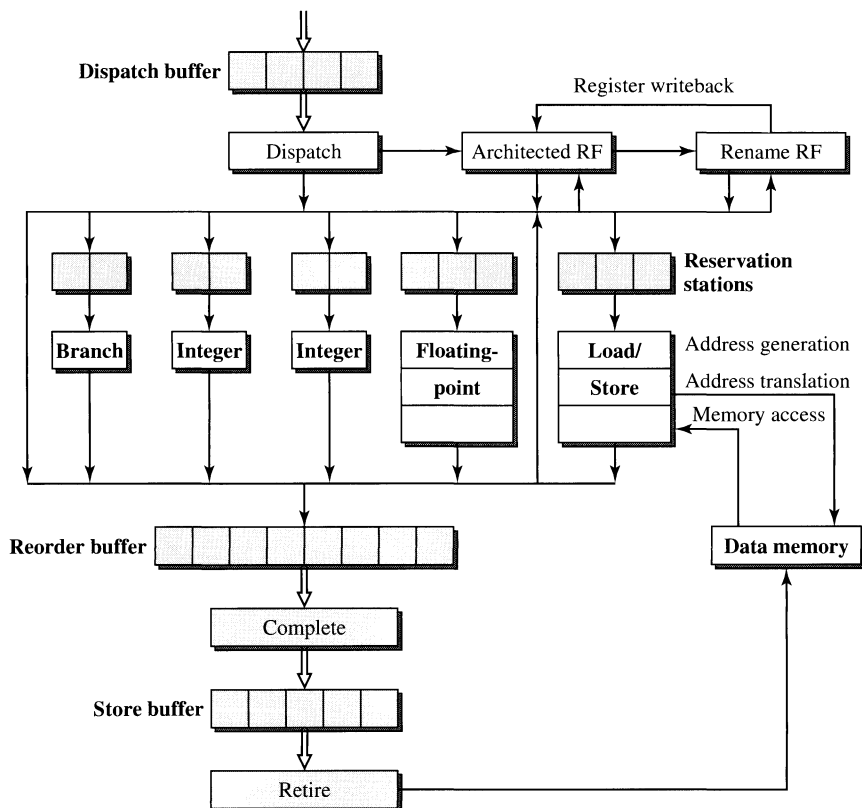


Figure 5.30
Processing of Load/Store Instructions.

After the first pipe stage generates the effective address, the second pipe stage translates this virtual address into a physical address. Typically, this is done by accessing the translation lookaside buffer (TLB), which is a hardware-controlled table containing the mapping of virtual to physical addresses. The TLB is essentially a cache of the page table that is stored in the main memory. (Section 3.6 provides more background material on the page table and the TLB.) It is possible that the virtual address being translated belongs to a page whose mapping is not currently resident in the TLB. This is called a *TLB miss*. If the particular mapping is present in the page table, then it can be retrieved by accessing the page table in the main memory. Once the missing mapping is retrieved and loaded into the TLB, the translation can be completed. It is also possible that the mapping is not resident even in the page table, meaning that the particular page being referenced has not been mapped and is not resident in the main memory. This will induce a *page fault* and require accessing disk storage to retrieve the missing page. This constitutes a program exception and will necessitate the suspension of the execution of the current program.

After successful address translation in the second pipe stage, a load instruction accesses the data memory during the third pipe stage. At the end of this machine cycle, the data are retrieved from the data memory and written into either the rename register or the reorder buffer. At this point the load instruction finishes execution. The updating of the architected register is not performed until this load instruction is completed from the reorder buffer. Here we assume that data memory access can be done in one machine cycle in the third pipe stage. This is possible if a data cache is employed. (Section 3.6 provides more background material on caches.) With a data cache, it is possible that the data being loaded are not resident in the data cache. This will result in a data *cache miss* and require the filling of the data cache from the main memory. Such cache misses can necessitate the stalling of the load/store pipeline.

Store instructions are processed somewhat differently than load instructions. Unlike a load instruction, a store instruction is considered as having *finished* execution at the end of the second pipe stage when there is a successful translation of the address. The register data to be stored to memory are kept in the reorder buffer. At the time when the store is being completed, these data are then written out to memory. The reason for this delayed access to memory is to prevent the premature and potentially erroneous update of the memory in case the store instruction may have to be flushed due to the occurrence of an exception or a branch misprediction. Since load instructions only read the memory, their flushing will not result in unwanted side effects to the memory state.

For a store instruction, instead of updating the memory at completion, it is possible to move the data to the store buffer at completion. The store buffer is a FIFO buffer that buffers architecturally completed store instructions. Each of these store instructions is then retired, i.e., updates the memory, when the memory bus is available. The purpose of the store buffer is to allow stores to be retired when the memory bus is not busy, thus giving priority to loads that need to access the memory bus. We use the term *completion* to refer to the updating of the CPU state and the term *retiring* to refer to the updating of the memory state. With the store buffer, a store instruction can be architecturally complete but not yet retired to memory.

When a program exception occurs, the instructions that follow the excepting instruction and that may have finished out of order, must be flushed from the reorder buffer; however, the store buffer must be drained, i.e., the store instructions in the store buffer must be retired, before the excepting program can be suspended.

We have assumed here that both address translation and memory accessing can be done in one machine cycle. Of course, this is only the case when both the TLB and the first level of the memory hierarchy return hits. An in-depth treatment of memory hierarchies that maximize the occurrence of cache hits by exploiting temporal and spatial locality and using various forms of caching is provided in Chapter 3. In Section 5.3.2, we will focus specifically on the additional complications that result from out-of-order execution of memory references and on some of the mechanisms used by modern processors to address these complications.

5.3.2 Ordering of Memory Accesses

A memory data dependence exists between two load/store instructions if they both reference the same memory location, i.e., there exists an *aliasing*, or collision, of the two memory addresses. A load instruction performs a read from a memory location, while a store instruction performs a write to a memory location. Similar to register data dependences, read-after-write (RAW), write-after-read (WAR), and write-after-write (WAW) dependences can exist between load and store instructions. A store (load) instruction followed by a load (store) instruction involving the same memory location will induce a RAW (WAR) memory data dependence. Two stores to the same memory location will induce a WAW dependence. These memory data dependences must be enforced in order to preserve the correct semantics of the program.

One way to enforce memory data dependences is to execute all load/store instructions in program order. Such total ordering of memory instructions is sufficient for enforcing memory data dependences but not necessary. It is conservative and can impose an unnecessary limitation on the performance of a program. We use the example in Figure 5.31 to illustrate this point. DAXPY is the name of a piece

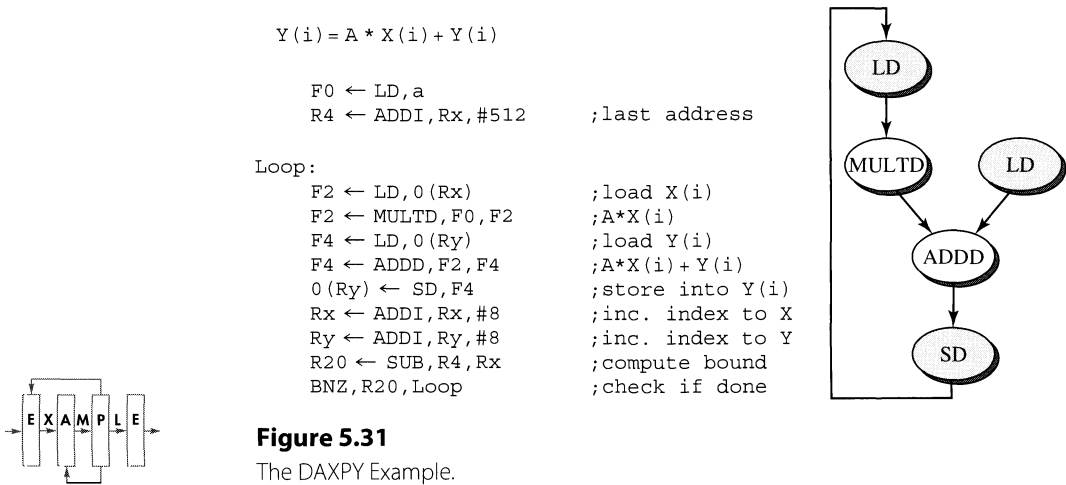


Figure 5.31
The DAXPY Example.

of code that multiplies an array by a coefficient and then adds the resultant array to another array. DAXPY (derived from “double precision A times X plus Y”) is a kernel in the LINPAC routines and is commonly found in many numerical programs. Notice that all the iterations of this loop are data-independent and can be executed in parallel. However, if we impose the constraint that all load/store instructions be executed in total order, then the first load instruction of the second iteration cannot begin until the store instruction of the first iteration is performed. This constraint will effectively serialize the execution of all the iterations of this loop.

By allowing load/store instructions to execute out of order, without violating memory data dependences, performance gain can be achieved. Take the example of the DAXPY loop. The graph in Figure 5.31 represents the true data dependences involving the core instructions of the loop body. These dependences exist among the instructions of the same iteration of the loop. There are no data dependences between multiple iterations of the loop. The loop closing branch instruction is highly predictable; hence, the fetching of subsequent iterations can be done very quickly. The same architected registers specified by instructions from subsequent iterations are dynamically renamed by register renaming mechanisms; hence, there are no register dependences between the iterations due to the dynamic reuse of the same architected registers. Consequently if load/store instructions are allowed to execute out of order, the load instructions from a trailing iteration can begin before the execution of the store instruction from an earlier iteration. By overlapping the execution of multiple iterations of the loop, performance gain is achieved for the execution of this loop.

Memory models impose certain limitations on the out-of-order execution of load/store instructions by a processor. First, to facilitate recovery from exceptions, the sequential state of the memory must be preserved. In other words, the memory state must evolve according to the sequential execution of load/store instructions. Second, many shared-memory multiprocessor systems assume the sequential consistency memory model, which requires that the accessing of the shared memory by each processor be done according to program order [Lamport, 1979, Adve and Gharachorloo, 1996].¹ Both of these reasons effectively require that store instructions be executed in program order, or at least the memory must be updated as if stores are performed in program order. If stores are required to execute in program order, WAW and WAR memory data dependences are implicitly enforced and are not an issue. Hence, only RAW memory data dependences must be enforced.

5.3.3 Load Bypassing and Load Forwarding

Out-of-order execution of load instructions is the primary source for potential performance gain. As can be seen in the DAXPY example, load instructions are frequently at the beginning of dependence chains, and their early execution can facilitate the early execution of other dependent instructions. While relative to memory data dependences, load instructions are viewed as performing read operations on the memory locations, they are actually performing write operations to their destination registers. With loads being register-defining (DEF) instructions, they

¹Consistency models are discussed in greater detail in Chapter 11.

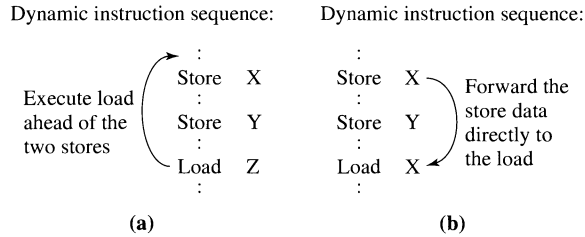


Figure 5.32

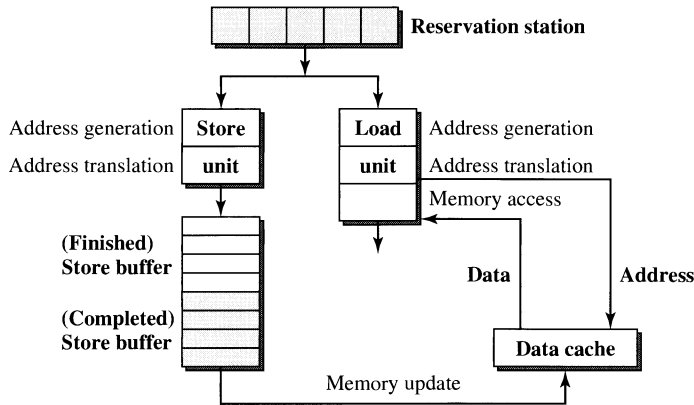
Early Execution of Load Instructions: (a) Load Bypassing;
(b) Load Forwarding.

are typically followed immediately by other dependent register use (USE) instructions. The goal is to allow load instructions to begin execution as early as possible, possibly jumping ahead of other preceding store instructions, as long as RAW memory data dependences are not violated and that memory is updated according to the sequential memory consistency model.

Two specific techniques for early out-of-order execution of loads are *load bypassing* and *load forwarding*. As shown in Figure 5.32(a), load bypassing allows a trailing load to be executed earlier than preceding stores if the load address does not alias with the preceding stores; i.e., there is no memory data dependence between the stores and the load. On the other hand, if a trailing load aliases with a preceding store, i.e., there is a RAW dependence from the store to the load, load forwarding allows the load to receive its data directly from the store without having to access the data memory; see Figure 5.32(b). In both of these cases, earlier execution of a load instruction is achieved.

Before we discuss the issues that must be addressed in order to implement load bypassing and load forwarding, first we present the organization of the portion of the execution core responsible for processing load/store instructions. This organization, shown in Figure 5.33, is used as the vehicle for our discussion on load bypassing and load forwarding. There is one store unit (two pipe stages) and one load unit (three pipe stages); both are fed by a common reservation station. For now we assume that load and store instructions are issued from this shared reservation station in program order. The store unit is supported by a store buffer. The load unit and the store buffer can access the data cache.

Given the organization of Figure 5.33, a store instruction can be in one of several states while it is in flight. When a store instruction is dispatched to the reservation station, an entry in the reorder buffer is allocated for it. It remains in the reservation station until all its source operands become available and it is issued into the pipelined execution unit. Once the memory address is generated and successfully translated, it is considered to have finished execution and is placed into the finished portion of the store buffer (the reorder buffer is also updated). The store buffer operates as a queue and has two portions, *finished* and *completed*. The finished portion contains those stores that have finished execution but are not yet architecturally

**Figure 5.33**

Mechanisms for Load/Store Processing: Separate Load and Store Units with In-Order Issuing from a Common Reservation Station.

completed. The completed portion of the store buffer contains those stores that are completed architecturally but waiting to update the memory. The identification of the two portions of the store buffer can be done via a pointer to the store buffer or a status bit in the store buffer entries. A store in the finished portion of the store buffer can potentially be speculative, and when a misspeculation is detected, it will need to be flushed from the store buffer. When a finished store is completed by the reorder buffer, it changes from the finished state to the completed state. This can be done by updating the store buffer pointer or flipping the status bit. When a completed store finally exits the store buffer and updates the memory, it is considered retired. Viewed from the perspective of the memory state, a store does not really finish its execution until it is retired. When an exception occurs, the stores in the completed portion of the store buffer must be drained in order to appropriately update the memory. So between being dispatched and retired, a store instruction can be in one of three states: *issued* (in the execution unit), *finished* (in the finished portion of the store buffer), or *completed* (in the completed portion of the store buffer).

One key issue in implementing load bypassing is the need to check for possible aliasing with preceding stores, i.e., those stores being bypassed. A load is considered to bypass a preceding store if the load reads from the memory before the store writes to the memory. Hence, before such a load is allowed to execute or read from the memory, it must be determined that it does not alias with all the preceding stores that are still in flight, i.e., those that have been issued but not retired. Assuming in-order issuing of load/store instructions from the load/store reservation station, all such stores should be sitting in the store buffer, including both the finished and the completed portions. The alias checking for possible dependence between the load and the preceding store can be done using the store buffer. A tag field containing the memory address of the store is incorporated with each entry of the store buffer. Once

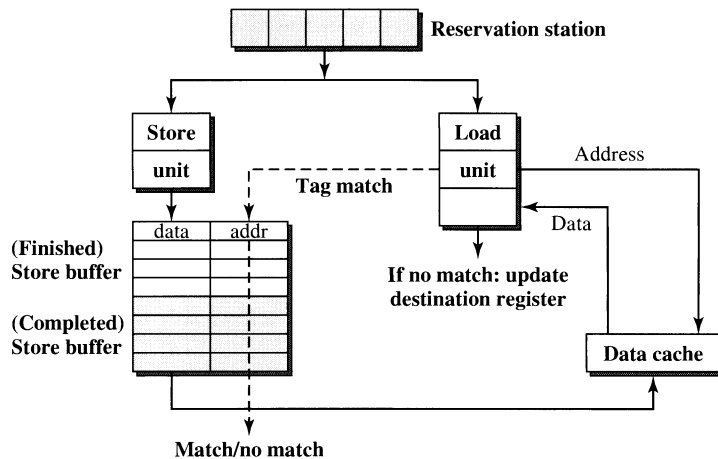
**Figure 5.34**

Illustration of Load Bypassing.

the memory address of the load is available, this address can be used to perform an associative search on the tag field of the store buffer entries. If a match occurs, then aliasing exists and the load is not allowed to execute out of order. Otherwise, the load is independent of the preceding stores in the store buffer and can be executed ahead of them. This associative search can be performed in the third pipe stage of the load unit concurrent with the accessing of the data cache; see Figure 5.34. If no aliasing is detected, the load is allowed to finish and the corresponding renamed destination register is updated with the data returned from the data cache. If aliasing is detected, the data returned by the data cache are discarded and the load is held back in the reservation station for future reissue.

Most of the complexity in implementing load bypassing lies in the store buffer and the associated associative search mechanism. To reduce the complexity, the tag field used for associative search can be reduced to contain only a subset of the address bits. Using only a subset of the address bits can reduce the width of the comparators needed for associative search. However, the result can be pessimistic. Potentially, an alias can be indicated by the narrower comparator when it really doesn't exist if the full-length address bits were used. Some of the load bypassing opportunities can be lost due to this compromise in the implementation. In general, the degradation of performance is minimal if enough address bits are used.

The load forwarding technique further enhances and complements the load bypassing technique. When a load is allowed to jump ahead of preceding stores, if it is determined that the load does alias with a preceding store, there is the potential to satisfy that load by forwarding the data directly from the aliased store. Essentially a memory RAW dependence exists between the leading store and the trailing load. The same associative search of the store buffer is needed. When aliasing is detected, instead of holding the load back for future reissue, the data from the aliased entry of the store buffer are forwarded to the renamed destination register

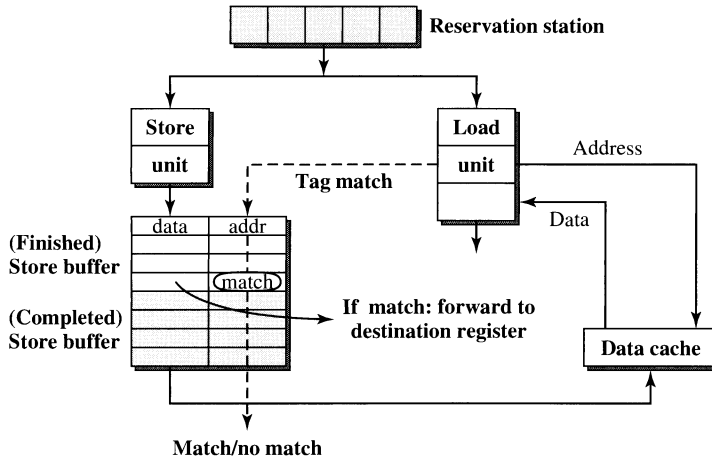
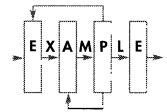


Figure 5.35

Illustration of Load Forwarding.



of the load instruction. This technique not only allows the load to be executed early, but also eliminates the need for the load to access the data cache. This can reduce the bandwidth pressure on the bus to the data cache.

To support load forwarding, added complexity to the store buffer is required; see Figure 5.35. First, the full-length address bits must be used for performing the associative search. When a subset of the address bits is used for supporting load bypassing, the only negative consequence is lost opportunity. For load forwarding the alias detection must be exact before forwarding of data can be performed; otherwise, it will lead to semantic incorrectness. Second, there can be multiple preceding stores in the store buffer that alias with the load. When such multiple matches occur during the associative search, there must be logic added to determine which of the aliased stores is the most recent. This will require additional priority encoding logic to identify the latest store on which the load is dependent before forwarding is performed. Third, an additional read port may be required for the store buffer. Prior to the incorporation of load forwarding, the store buffer has one write port that interfaces with the store unit and one read port that interfaces with the data cache. A new read port is now required that interfaces with the load unit; otherwise, port contention can occur between load forwarding and data cache update.

Significant performance improvement can be obtained with load bypassing and load forwarding. According to Mike Johnson, typically load bypassing can yield 11% to 19% performance gain, and load forwarding can yield another 1% to 4% of additional performance improvement [Johnson, 1991].

So far we have assumed that loads and stores share a common reservation station with instructions being issued from the reservation station to the store and the load units in program order. This in-order issuing assumption ensures that all the preceding stores to a load will be in the store buffer when the load is executed. This simplifies memory dependence checking; only an associative search of the

store buffer is necessary. However, this in-order issuing assumption introduces an unnecessary limitation on the out-of-order execution of loads. A load instruction can be ready to be issued; however, a preceding store can hold up the issuing of the load even though the two memory instructions do not alias. Hence, allowing out-of-order issuing of loads and stores from the load/store reservation station can permit a greater degree of out-of-order and early execution of loads. This is especially beneficial if these loads are at the beginnings of critical dependence chains and their early execution can remove critical performance bottlenecks.

If out-of-order issuing from the load/store reservation station is allowed, a new problem must be solved. If a load is allowed to be issued out of order, then it is possible for some of the stores that precede it to still be in the reservation station or in the execution pipe stages, and not yet in the store buffer. Hence, simply performing an associative search on the entries of the store buffer is not adequate for checking for potential aliasing between the load and all its preceding stores. Worse yet, the memory addresses for these preceding stores that are still in the reservation station or in the execution pipe stages may not be available yet.

One approach is to allow the load to proceed, assuming no aliasing with the preceding stores that are not yet in the store buffer, and then validate this assumption later. With this approach, a load is allowed to issue out of order and be executed speculatively. If it does not alias with any of the stores in the store buffer, the load is allowed to finish execution. However, this load must be put into a new buffer called the finished load buffer; see Figure 5.36. The finished load buffer is managed in a similar fashion as the finished store buffer. A load is only resident in the finished load buffer after it finishes execution and before it is completed.

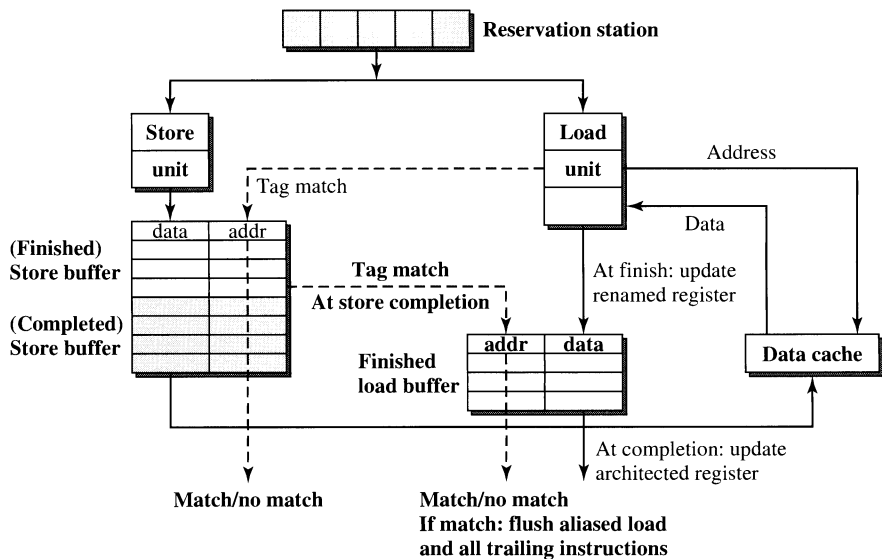


Figure 5.36

Fully Out-of-Order Issuing and Execution of Load and Store Instructions.

Whenever a store instruction is being completed, it must perform alias checking against the loads in the finished load buffer. If no aliasing is detected, the store is allowed to complete. If aliasing is detected, then it means that there is a trailing load that is dependent on the store, and that load has already finished execution. This implies that the speculative execution of that load must be invalidated and corrected by reissuing, or even refetching, that load and all subsequent instructions. This can require significant hardware complexity and performance penalty.

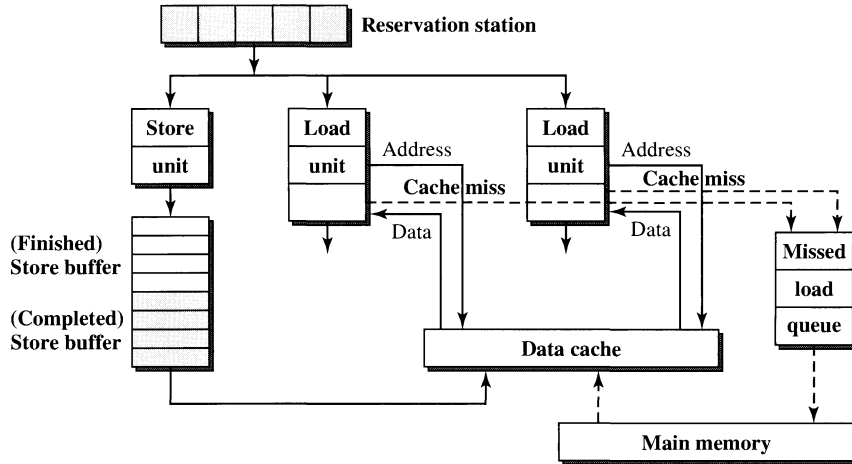
Aggressive early issuing of load instructions can lead to significant performance benefits. The ability to speculatively issue loads ahead of stores can lead to early execution of many dependent instructions, some of which can be other loads. This is important especially when there are cache misses. Early issuing of loads can lead to early triggering of cache misses which can in turn mask some or all of the cache miss penalty cycles. The downside with speculative issuing of loads is the potential overhead of having to recover from misspeculation. One way to reduce this overhead is to do alias or *dependence prediction*. In typical programs, the dependence relationship between a load and its preceding stores is quite predictable. A memory dependence predictor can be implemented to predict whether a load is likely to alias with its preceding stores. Such a predictor can be used to determine whether to speculatively issue a load or not. To obtain actual performance gain, aggressive speculative issuing of loads must be done very judiciously.

Moshovos [1998] proposed a memory dependence predictor that was used to directly bypass store data to dependent loads via memory cloaking. In subsequent work, Chrysos and Emer [1998] described a similar predictor called the store-set predictor.

5.3.4 Other Memory Data Flow Techniques

Other than load bypassing and load forwarding, there are other memory data flow techniques. These techniques all have the objectives of increasing the memory bandwidth and/or reducing the memory latency. As superscalar processors get wider, greater memory bandwidth capable of supporting multiple load/store instructions per cycle will be needed. As the disparity between processor speed and memory speed continues to increase, the latency of accessing memory, especially when cache misses occur, will become a serious bottleneck to machine performance. Sohi and Franklin [1991] studied high-bandwidth data memory systems.

One way to increase memory bandwidth is to employ multiple load/store units in the execution core, supported by a multiported data cache. In Section 5.3.3 we have assumed the presence of one store unit and one load unit supported by a single-ported data cache. The load unit has priority in accessing the data cache. Store instructions are queued in the store buffer and are retired from the store buffer to the data cache whenever the memory bus is not busy and the store buffer can gain access to the data cache. The overall data memory bandwidth is limited to one load/store instruction per cycle. This is a serious limitation, especially when there are bursts of load instructions. One way to alleviate this bottleneck is to provide two load units, as shown in Figure 5.37, and a *dual-ported data cache*. A dual-ported data cache is able to support two simultaneous cache accesses in

**Figure 5.37**

Dual-Ported and Nonblocking Data Cache.

every cycle. This will double the potential memory bandwidth. However, it comes with the cost of hardware complexity; a dual-ported cache can require doubling of the cache hardware. One way to alleviate this hardware cost is to implement interleaved data cache banks. With the data cache being implemented as multiple banks of memory, two simultaneous accesses to different banks can be supported in one cycle. If two accesses need to access the same bank, a bank conflict occurs and the two accesses must be serialized. From practical experience, a cache with eight banks can keep the frequency of bank conflicts down to acceptable levels.

The most common way to reduce memory latency is through the use of a cache. Caches are now widely employed. As the gap between processor speed and memory speed widens, multiple levels of caches are required. Most high-performance superscalar processors incorporate at least two levels of caches. The first level (L1) cache can usually keep up with the processor speed with access latency of one or very few cycles. Typically there are separate L1 caches for storing instructions and data. The second level (L2) cache typically supports the storing of both instructions and data, can be either on-chip or off-chip, and can be accessed in series (in case of a miss in the L1) or in parallel with the L1 cache. In some of the emerging designs a third level (L3) cache is used. It is likely that in future high-end designs a very large on-chip L3 cache will become commonplace. Other than the use of a cache or a hierarchy of caches, there are two other techniques for reducing the effective memory latency, namely, *nonblocking cache* and *prefetching cache*.

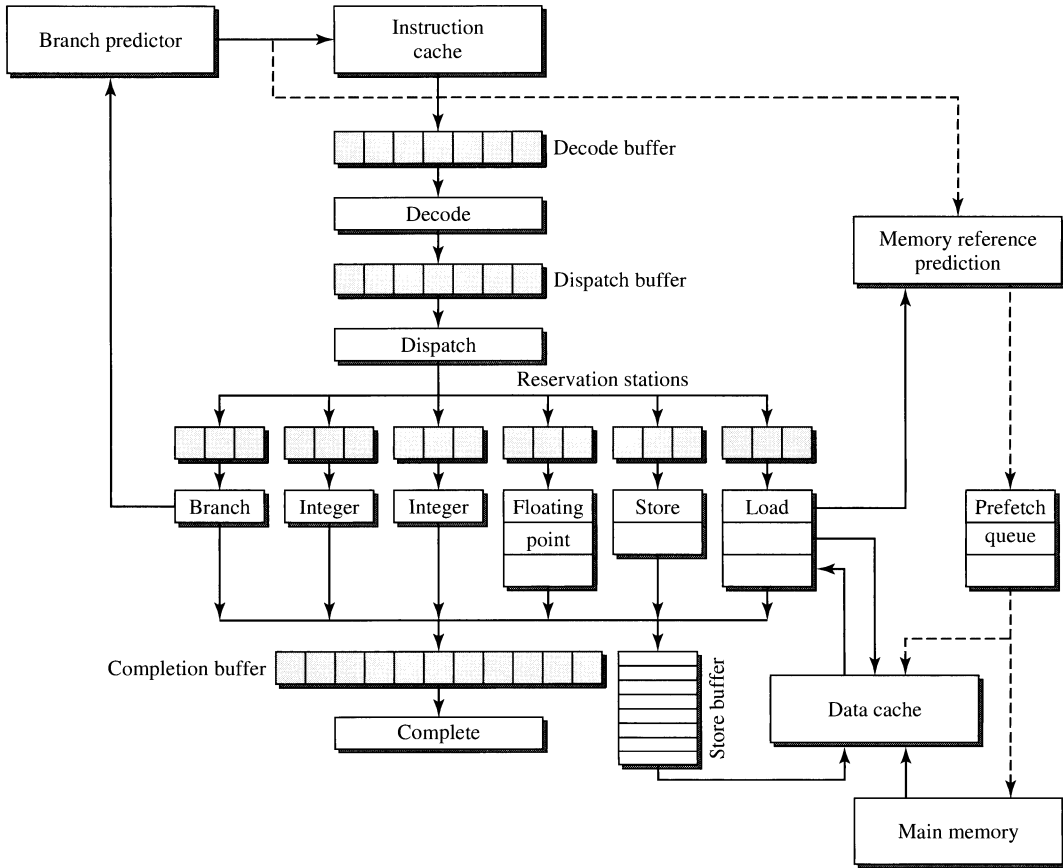
A nonblocking cache, first proposed by Kroft [1981], can reduce the effective memory latency by reducing the performance penalty due to cache misses. Traditionally, when a load instruction encounters a cache miss, it will stall the load unit pipeline and any further issuing of load instructions until the cache miss is serviced.

Such stalling is overly conservative and prevents subsequent and independent loads that may hit in the data cache from being issued. A nonblocking data cache alleviates this unnecessary penalty by putting aside a load that has missed in the cache into a *missed load queue* and allowing subsequent load instructions to issue; see Figure 5.37. A missed load sits in the missed load queue while the cache miss is serviced. When the missing block is fetched from the main memory, the missed load exits the missed load queue and finishes execution.

Essentially the cache miss penalty cycles are overlapped with, and masked by, the processing of subsequent independent instructions. Of course, if a subsequent instruction depends on the missed load, the issuing of that instruction is stalled. The number of penalty cycles that can be masked depends on the number of independent instructions following the missed load. A missed load queue can contain multiple entries, allowing multiple missed loads to be serviced concurrently. Potentially the cache penalty cycles of multiple missed loads can be overlapped to result in fewer total penalty cycles.

A number of issues must be considered when implementing nonblocking caches. Load misses can occur in bursts. The ability to support multiple misses and overlap their servicing is important. The interface to main memory, or a lower-level cache, must be able to support the overlapping or pipelining of multiple accesses. The filling of the cache triggered by the missed load may need to contend with the store buffer for the write port to the cache. There is one complication that can emerge with nonblocking caches. If the missed load is on a speculative path, i.e., the predicted path, there is the possibility that the speculation, i.e., branch prediction, will turn out to be incorrect. If a missed load is on a mispredicted path, the question is whether the cache miss should be serviced. In a machine with very aggressive branch prediction, the number of loads on the mispredicted path can be significant; servicing their misses speculatively can require significant memory bandwidth. Studies have shown that a nonblocking cache can reduce the amount of load miss penalty by about 15%.

Another way to reduce or mask the cache miss penalty is through the use of a prefetching cache. A prefetching cache anticipates future misses and triggers these misses early so as to overlap the miss penalty with the processing of instructions preceding the missing load. Figure 5.38 illustrates a prefetching data cache. Two structures are needed to implement a prefetching cache, namely, a *memory reference prediction table* and a *prefetch queue*. The memory reference prediction table stores information about previously executed loads in three different fields. The first field contains the instruction address of the load and is used as a tag field for selecting an entry of the table. The second field contains the previous data address of the load, while the third field contains a stride value that indicates the difference between the previous two data addresses used by that load. The memory reference prediction table is accessed via associative search using the fetch address produced by the branch predictor and the first field of the table. When there is a tag match, indicating a hit in the memory reference prediction table, the previous address is added to the stride value to produce a predicted memory address. This predicted address is then loaded into the prefetch queue. Entries in the prefetch queue are

**Figure 5.38**

Prefetching Data Cache.

retrieved to speculatively access the data cache, and if a cache miss is triggered, the main memory or the next-level cache is accessed. The access to the data cache is in reality a cache touch operation; i.e., access to the cache is attempted in order to trigger a potential cache miss and not necessarily to actually retrieve the data from the cache. Early work on prefetching was done by Baer and Chen [1991] and Jouppi [1990].

The goal of a prefetching cache is to try to anticipate forthcoming cache misses and to trigger those misses early so as to hide the cache miss penalty by overlapping cache refill time with the processing of instructions preceding the missing load. When the anticipated missing load is executed, the data will be resident in the cache; hence, no cache miss is triggered, and no cache miss penalty is incurred. The actual effectiveness of prefetching depends on a number of factors. The prefetching distance, i.e., how far in advance the prefetching is being triggered, must be large enough to fully mask the miss penalty. This is the reason that

the predicted instruction fetch address is used to access the memory reference prediction table, with the hope that the data prefetch will occur far enough in advance of the load. However, this makes prefetching effectiveness subject to the effectiveness of branch prediction. Furthermore, there is the potential of polluting the data cache with prefetches that are on the mispredicted path. Status or confidence bits can be added to each entry of the memory reference prediction table to modulate the aggressiveness of prefetching. Another problem can occur when the prefetching is performed too early so as to evict a useful block from the cache and induce an unnecessary miss. One more factor is the actual memory reference prediction algorithm used. Load address prediction based on stride is quite effective for loads that are stepping through an array. For other loads that are traversing linked list data structures, the stride prediction will not work very well. Prefetching for such memory references will require much more sophisticated prediction algorithms.

To enhance memory data flow, load instructions must be executed early and fast. Store instructions are less important because experimental data indicate that they occur less frequently than load instructions and they usually are not on the performance critical path. To speed up the execution of loads we must reduce the latency for processing load instructions. The overall latency for processing a load instruction includes four components: (1) pipeline front-end latency for fetching, decoding, and dispatching the load instruction; (2) reservation station latency of waiting for register data dependence to resolve; (3) execution pipeline latency for address generation and translation; and (4) the cache/memory access latency for retrieving the data from memory. Both nonblocking and prefetching caches address only the fourth component, which is a crucial component due to the slow memory speed. To achieve higher clocking rates, superscalar pipelines are quickly becoming deeper and deeper. Consequently the latencies, in terms of number of machine cycles, of the first three components are also becoming quite significant. A number of speculative techniques have been proposed to address the reduction of these latencies; they include load address prediction, load value prediction, and memory dependence prediction.

Recently *load address prediction* techniques have been proposed to address the latencies associated with the first three components [Austin and Sohi, 1995]. To deal with the latency associated with the first component, a load prediction table, similar to the memory reference prediction table, is proposed. This table is indexed with the predicted instruction fetch address, and a hit in this table indicates the presence of a load instruction in the upcoming fetch group. Hence, the prediction of the presence of a load instruction in the upcoming fetch group is performed during the fetch stage and without requiring the decode and dispatch stages. Each entry of this table contains the predicted effective address which is retrieved during the fetch stage, in effect eliminating the need for waiting in the reservation station for the availability of the base register value and the address generation stage of the execution pipeline. Consequently, data cache access can begin in the next cycle, and potentially data can be retrieved from the cache at the end of the decode stage. Such a form of load address prediction can effectively collapse the latencies of the first three components down to just two cycles, i.e.,

fetch and decode stages, if the address prediction is correct and there is a hit in the data cache.

While the hardware structures needed to support load address prediction are quite similar to those needed for memory prefetching, the two mechanisms have significant differences. Load address prediction is actually executing, though speculatively, the load instruction early, whereas memory prefetching is mainly trying to prefetch the needed data into the cache without actually executing the load instruction. With load address prediction, instructions that depend on the load can also execute early because their dependent data are available early. Since load address prediction is a speculative technique, it must be validated, and if misprediction is detected, recovery must be performed. The validation is performed by allowing the actual load instruction to be fetched from the instruction cache and executed in a normal fashion. The result from the speculative version is compared with that of the normal version. If the results concur, then the speculative result becomes nonspeculative and all the dependent instructions that were executed speculatively are also declared as nonspeculative. If the results do not agree, then the nonspeculative result is used and all dependent instructions must be reexecuted. If the load address prediction mechanism is quite accurate, mispredictions occur only infrequently, the misprediction penalty is minimal, and overall net performance gain can be achieved.

Even more aggressive than load address prediction is the technique of *load value prediction* [Lipasti et al., 1996]. Unlike load address prediction which attempts to predict the effective address of a load instruction, load value prediction actually attempts to predict the value of the data to be retrieved from memory. This is accomplished by extending the load prediction table to contain not just the predicted address, but also the predicted value for the destination register. Experimental studies have shown that many load instructions' destination values are quite predictable. For example, many loads actually load the same value as last time. Hence, by storing the last value loaded by a static load instruction in the load prediction table, this value can be used as the predicted value when the same static load is encountered again. As a result, the load prediction table can be accessed during the fetch stage, and at the end of that cycle, the actual destination value of a predicted load instruction can be available and used in the next cycle by a dependent instruction. This significantly reduces the latency required for processing a load instruction if the load value prediction is correct. Again, validation is required, and at times a misprediction penalty must be paid.

Other than load address prediction and load value prediction, a third speculative technique has been proposed called *memory dependence prediction* [Moshovos, 1998]. Recall from Section 5.3.3 that to perform load bypassing and load forwarding, memory dependence checking is required. For load bypassing, it must be determined that the load does not alias with any of the stores being bypassed. For load forwarding, the most recent aliased store must be identified. Memory dependence checking can become quite complex if a larger number of load/store instructions are involved and can potentially require an entire pipe stage. It would be nice to eliminate this latency. Experimental data have shown

that the memory dependence relationship is quite predictable. It is possible to track the memory dependence relationship when load/store instructions are executed and use this information to make memory dependence prediction when the same load/store instructions are encountered again. Such memory dependence prediction can facilitate earlier execution of load bypassing and load forwarding. As with all speculative techniques, validation is needed and a recovery mechanism for misprediction must be provided.

5.4 Summary

In this chapter we attempt to cover all the fundamental microarchitecture techniques used in modern superscalar microprocessor design in a systematic and easy-to-digest way. We intentionally avoid inundating readers with lots of quantitative data and bar charts. We also focus on generic techniques instead of features of specific commercial products. In Chapters 6 and 7 we present detailed case studies of two actual products. Chapter 6 introduces the IBM/Motorola PowerPC 620 in great detail along with quantitative performance data. While not a successful commercial product, the PowerPC 620 represents one of the earlier and most aggressive out-of-order designs. Chapter 7 presents the Intel P6 microarchitecture, the first out-of-order implementation of the IA32 architecture. The Intel P6 is likely the most commercially successful microarchitecture. The fact that the P6 microarchitecture core provided the foundation for multiple generations of products, including the Pentium Pro, the Pentium II, the Pentium III, and the Pentium M, is a clear testimony to the effectiveness and elegance of its original design.

Superscalar microprocessor design is a rapidly evolving art which has been simultaneously harnessing the creative ideas of researchers and the insights and skills of architects and designers. Chapter 8 is a historical chronicle of the practice of this art form. Interesting and valuable lessons can be gleaned from this historical chronicle. The body of knowledge on superscalar microarchitecture techniques is constantly expanding. New innovative ideas from the research community as well as ideas from the traditional “macroarchitecture” domain are likely to find their way into future superscalar microprocessor designs. Chapters 9, 10, and 11 document some of these ideas.

REFERENCES

- Adve, S. V., and K. Gharachorloo: “Shared memory consistency models: A tutorial,” *IEEE Computer*, 29, 12, 1996, pp. 66–76.
- Austin, T. M., and G. S. Sohi: “Zero-cycle loads: Microarchitecture support for reducing load latency,” *Proc. 28th Annual ACM/IEEE Int. Symposium on Microarchitecture*, 1995, pp. 82–92.
- Baer, J., and T. Chen: “An effective on-chip preloading scheme to reduce data access penalty,” *Proc. Supercomputing '91*, 1991, pp. 176–186.
- Calder, B., P. Feller, and A. Eustace: “Value profiling,” *Proc. 30th Annual ACM/IEEE Int. Symposium on Microarchitecture*, 1997, pp. 259–269.